



Proyecto 1 - Airbnb DBMS

Bases de Datos 2



Integrantes:

Laura Gabriela Nagamine Oshiro
Sofia Valentina Ku Paredes
Hanks Jean Pierce Vargas Iglesias
Luis Anthony Romero Padilla
Maria Karolay Tamayo Hilario

Ciclo 2026-1

Lima, Perú

Índice

1. Introducción	3
2. Especificación funcional	3
2.1. Flujo general de la base de datos	3
2.2. Gestores de Base de Datos	5
2.2.1. Engine	5
2.2.2. DB Manager	6
2.2.3. File Manager	9
2.2.4. Buffer Manager	10
2.3. Estructuras de indexación sobre memoria secundaria	11
2.3.1. Sequential File	11
2.3.2. Extendible Hashing	13
2.3.3. B+ Tree	16
2.3.4. R-Tree	20
2.4. Parser SQL personalizado	26
2.4.1. Gramática	26
2.4.2. Scanner	27
2.4.3. Parser	27
3. Consideraciones técnicas	29
4. Simulador de acceso concurrente	29
4.1. Control de concurrencia: decisiones de implementación	29
4.2. Justificación de las decisiones de diseño	30
4.3. Archivos modificados y creados	31
4.3.1. indexes/deadlock_detector.py	31
4.3.2. indexes/rw_lock.py	31
4.3.3. indexes/buffer_manager.py	31
4.3.4. manager/record_file.py	32
4.3.5. manager/transaction_log.py	32
4.3.6. manager/db_manager.py	32
4.3.7. simulador_concurrencia.py	32
5. Interfaz gráfica de usuario	33

5.1. Componentes principales	33
5.2. Funcionalidades	33
6. Evaluación experimental	34
6.1. Metodología	34
6.2. Resultados experimentales	35
6.2.1. Inserción	35
6.2.2. Búsqueda puntual	40
6.2.3. Búsqueda por rango	44
6.2.4. DELETE	47
6.2.5. R-Tree – Consultas espaciales	51
6.3. Discusión de resultados	54
7. Pruebas de uso e interfaz gráfica	55
8. Algoritmos de memoria externa	56
8.1. External Sort	56
8.2. External Hashing para Join	56
8.3. Group By y Count(*)	59
8.3.1. Gramática soportada	59
8.3.2. Cambios por capa	60
8.3.3. Flujo de ejecución end-to-end	62
8.4. Replacement Selection	63
9. Conclusiones	64
10. Referencias	64

1. Introducción

El presente proyecto implementa un motor de base de datos relacional construido completamente sobre archivos binarios y estructuras de indexación en memoria secundaria, desarrollado íntegramente en Python. El sistema es capaz de ejecutar consultas SQL sobre tablas definidas por el usuario, soportando múltiples estructuras de índices, manejo de buffer en memoria y algoritmos de memoria externa.

El objetivo principal del proyecto es diseñar e implementar un sistema capaz de simular el funcionamiento interno de un DBMS real, integrando componentes como parser SQL, administración de páginas, gestión de índices y algoritmos de acceso eficiente a disco. Asimismo, se busca analizar experimentalmente el comportamiento de distintas técnicas de indexación sobre memoria secundaria y evaluar su impacto sobre el rendimiento de operaciones de inserción, búsqueda y eliminación.

Se utilizó el dataset Airbnb Open Data de Kaggle (Metawea, 2026), el cual contiene información de alojamientos de Nueva York, incluyendo atributos como precio, tipo de habitación, ubicación geográfica, identificadores y vecindarios. El dataset fue limpiado y transformado para soportar estructuras espaciales mediante columnas POINT construidas a partir de latitud y longitud.

El sistema implementa múltiples estructuras de indexación como Sequential File, B+ Tree, Extendible Hashing y R-Tree, además de un parser SQL personalizado, Buffer Manager con política LRU y algoritmos de memoria externa como External Sort y External Hashing Join.

Los resultados esperados incluyen la evaluación comparativa del desempeño de las distintas técnicas de indexación en términos de accesos a disco y tiempo de ejecución, así como demostrar el correcto funcionamiento del motor mediante consultas SQL y búsquedas espaciales.

2. Especificación funcional

2.1. Flujo general de la base de datos

En la arquitectura propuesta, se quiso replicar el flujo de ejecución de un DBMS real, separando responsabilidades entre parsing, ejecución lógica, administración de almacenamiento y gestión de índices.

El flujo general inicia cuando el usuario escribe una consulta SQL desde la interfaz gráfica. Esta consulta es enviada al backend mediante una API y procesada por el motor de ejecución donde pasa por Scanner, Parser, Engine, DBManager y finalmente las estructuras físicas de almacenamiento.

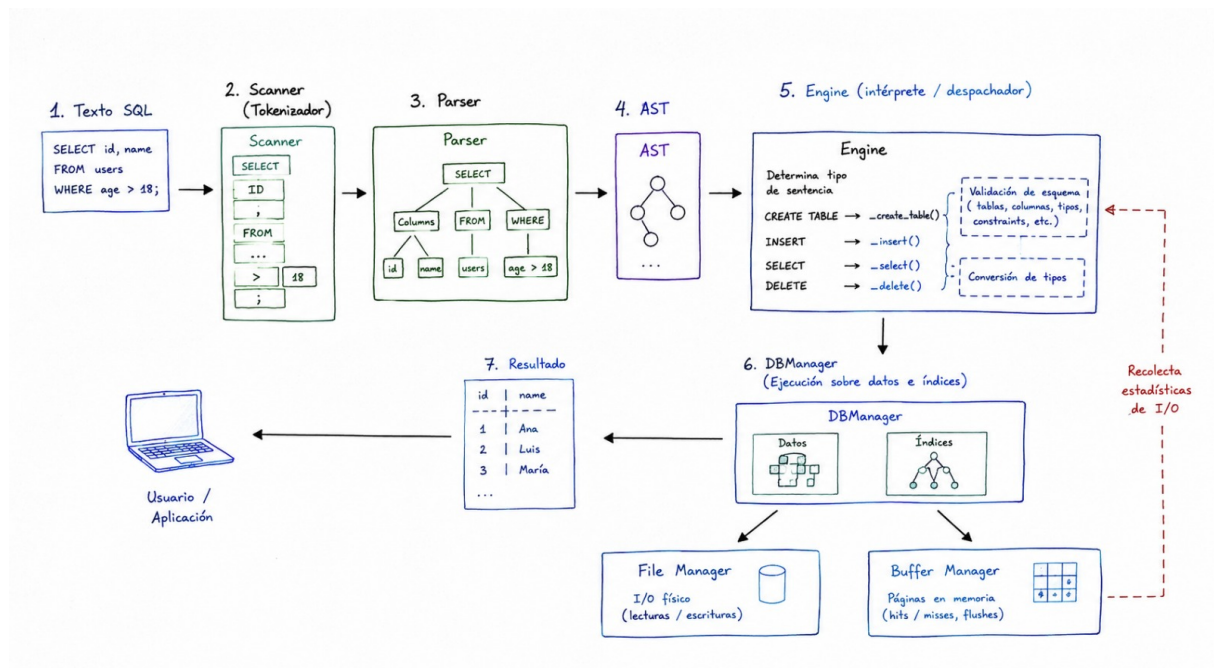


Figura 1: Flujo general del sistema

Se comienza por el Scanner o tokenizador, cuya función es transformar el texto SQL en una secuencia de tokens reconocibles por el parser. Luego, el Parser procesa estos tokens y construye un Abstract Syntax Tree (AST) que representa la estructura lógica de la consulta. El AST es luego procesado por Engine. Este interpreta el tipo de sentencia recibida, ejecuta la operación correspondiente, valida tipos, detecta errores semánticos y recolecta estadísticas de I/O. Básicamente, Engine funciona como coordinador central del sistema. Después de esta etapa, se pasa al DBManager, la capa de administración de datos e índices. El DBManager traduce las operaciones lógicas en acciones físicas sobre archivos binarios. Finalmente, las operaciones físicas son realizadas por el File Manager y el Buffer Manager. El File Manager administra lecturas y escrituras sobre páginas de 4096 bytes, mientras que el Buffer Manager mantiene páginas en memoria utilizando política LRU y registra métricas como hits, misses y flushes. El resultado de la consulta, junto con las estadísticas de ejecución, es retornado nuevamente al frontend.

Cuadro 1: Flujo general del sistema

Etapas	Componente	Responsabilidad
1	Query SQL	Consulta escrita por el usuario
2	Scanner	Convierte el texto SQL en tokens
3	Parser	Construye el AST
4	AST	Representa la consulta en una estructura procesable
5	Engine	Valida, interpreta y despacha operaciones
6	DBManager	Ejecuta operaciones sobre tablas, registros e índices
7	File Manager	Realiza lecturas y escrituras físicas sobre páginas
8	Buffer Manager	Mantiene páginas en memoria y registra hits/misses
9	Resultado	Devuelve resultados y estadísticas al frontend

2.2. Gestores de Base de Datos

2.2.1. Engine

Engine es el componente encargado de coordinar todo el flujo de ejecución del sistema. Su responsabilidad principal es interpretar las sentencias SQL ya parseadas y despachar las operaciones correspondientes hacia el DBManager.

El flujo general ejecutado por Engine es el siguiente:

1. El texto SQL es enviado al Scanner para generar tokens.
2. Los tokens son procesados por el Parser para construir el AST.
3. Engine interpreta el tipo de sentencia.
4. Se ejecuta el método correspondiente:
 - `_create_table()`
 - `_insert()`
 - `_select()`
 - `_delete()`
5. Se validan tipos y esquemas.
6. El DBManager ejecuta las operaciones físicas.
7. Finalmente, Engine recolecta estadísticas de I/O.

Las estadísticas retornadas por consulta incluyen: lecturas y escrituras a disco, hits y misses del buffer, porcentaje de uso del pool, y el tiempo de ejecución en milisegundos. Esta información se muestra en la interfaz gráfica en tiempo real.

2.2.2. DB Manager

DB Manager es el componente encargado de conectar la representación lógica de una tabla con su almacenamiento físico en disco. Este recibe operaciones ya interpretadas por el Engine y las traduce en acciones físicas sobre archivos binarios e índices.

Su función principal es centralizar la administración de tablas, registros e índices, actuando como una capa intermedia entre la lógica SQL y las estructuras físicas del sistema.

Organización física

Cada tabla se almacena dentro de:

```
data/tables/<tabla>
```

La información se divide en dos archivos principales: metadata.dat y records.dat.

En el primero, se guarda el esquema de la tabla serializada utilizando pickle, en el cual, almacena información lógica de la tabla tales como nombre de la tabla, columnas, tipos de dato, tamaño de strings, llave primaria e índice asociado a cada columna.

El segundo, guarda los registros completos en formato binario paginado.

Los índices son almacenados en:

```
data/indexes/<tabla>/<indice>
```

Esta separación entre datos e índices replica el comportamiento de motores reales y evita acoplar el almacenamiento principal con una estructura de acceso específica.

Gestión de esquemas

El DBManager utiliza las clases definidas en `schemas.py` para representar la estructura lógica de las tablas. Este módulo es fundamental porque establece el formato interno que utilizarán posteriormente el DBManager, RecordFile y los índices.

Cuadro 2: Clases de schemas.py

Concepto	Descripción
DataType	Soporta cinco tipos principales de data: INT, FLOAT, STRING, BOOL, POINT. Esto ayudará al cálculo de offsets físicos y permite acceder directamente a registros dentro de los archivos binarios sin necesidad de recorrer secuencialmente
IndexType	Soporta NONE, SEQFILE, EXTHASH, BTREE y RTREE
Column	Representa una columna individual dentro de una tabla. La información que almacena es nombre de la columna, indicador si es llave primaria, su data type, su index type, y el tamaño para columnas string.
Table	Representa el esquema lógico completo de una tabla. Esta guarda el nombre de la tabla y un arreglo de columnas. Sus funciones principales son get_column(), el cual retorna una columna específica según su nombre, y get_column_index(), el cual retorna la posición de una columna dentro de la tabla.
Record	Representa un registro individual almacenado físicamente en records.dat. La información que almacena es referencia al esquema table, valores del registro, id, formato binario calculado, tamaño físico del registro. Sus funciones principales son pack(), que convierte el registro desde representación lógica a binaria, y unpack(), que reconstruye el objeto Record interpretando el formato binario.
calc_record_format	Función auxiliar que genera el formato binario de un registro a partir de las columnas de la tabla. En el caso de strings, se establece un máximo de 20 caracteres.

Record File

La clase RecordFile implementa el almacenamiento físico principal de los registros.

Cada tabla posee un único archivo:

`records.dat`

dividido en páginas de 4096 bytes.

Las funciones principales son:

Cuadro 3: Funciones principales de RecordFile

Función	Descripción
add()	Inserta un nuevo registro dentro del archivo. Primero, obtiene next_record_id y calcula página y slot. Luego serializa el registro. Por último, escribe el slot en disco y actualiza next_record_id. Se retorna el record_id del record actual.
read()	Lee un registro activo a partir de un record_id. Este verifica la existencia del registro, valida que no esté eliminado, desempaqueta el slot y retorna el registro en el objeto Record.
delete()	Implementa borrado lógico. El registro no se elimina físicamente, solo se marca el slot como eliminado. Esto mantiene estable los record_id, evitando invalidar índices.
scan_all()	Recorre secuencialmente todos los registros activos de la tabla. Se utiliza principalmente cuando se necesita scans completos, queries con SELECT o joins.
record_count()	Cuenta la cantidad de registros válidos dentro del archivo. Se utiliza principalmente para estadísticas, joins y planificación de operaciones.
_record_offset()	Calcula offsets físicos de un registro.
_pack_slot()	Convierte registros a formato binario.
_unpack_slot()	Reconstruye registros desde bytes.

Operaciones principales del DBManager

Cuadro 4: Funciones principales del DBManager

Función	Descripción
<code>create_table()</code>	Crea una nueva tabla dentro del sistema. Primero valida el esquema y, si existe, elimina el almacenamiento previo. Después guarda la metadata, crea Record File e índices. Por último, carga CSV si aplica el caso.
<code>validate_table()</code>	Verifica que la tabla tenga nombre, al menos una columna, no pueden existir columnas duplicadas, sólo puede existir un primary key, string debe tener tamaño positivo, point solo puede usar índice R-Tree, R-Tree solo puede aplicarse a tipo de data POINT.
<code>insert()</code>	Inserta una fila dentro de la tabla. Primero valida el tipo y si es primary key. Luego crea un Record y lo inserta en el archivo correspondiente. Por último, actualiza los índices.
<code>select_equal()</code>	Ejecuta la búsqueda por igualdad. Esta búsqueda se realiza en el documento correspondiente de índice. Dependiendo del índice se realiza un tipo de búsqueda específica, para el extendible hash se hace una búsqueda hash; btree, búsqueda puntual; sequential file, búsqueda exacta o rango; none, scan completo.
<code>select_range()</code>	Ejecuta consultas BETWEEN para BTree y Sequential File, no para Extendible Hashing.
<code>delete_where_equal()</code>	Elimina registros que cumplen una condición.
<code>select_spatial_radius()</code>	Ejecuta búsquedas espaciales por radio.
<code>select_spatial_knn()</code>	Ejecuta búsquedas KNN.
<code>sort_by()</code>	Ejecuta External Sort
<code>external_hashing_join()</code>	Ejecuta joins mediante hashing externo

2.2.3. File Manager

El File Manager es la capa de más bajo nivel del sistema, encargada de toda la entrada y salida con el disco. Este componente abstrae el acceso a archivos mediante el concepto de páginas de tamaño fijo de 4096 bytes. De esta manera, las estructuras superiores no trabajan directamente con offsets arbitrarios del archivo, sino con identificadores de página.

Sus operaciones principales son:

Cuadro 5: Operaciones principales del File Manager

Función	Descripción
<code>read_page(path, page_id)</code>	Lee una página de 4096 bytes desde la posición <code>page_id * PAGE_SIZE</code> del archivo.
<code>write_page(path, page_id, data)</code>	Escribe una página completa en la posición indicada. Si el archivo no tiene suficiente tamaño, se extiende para permitir la escritura.
<code>append_page(path, data)</code>	Agrega una nueva página al final del archivo y retorna el nuevo <code>page_id</code> .
<code>read_bytes()</code> / <code>write_bytes()</code>	Permiten acceso a bytes en posiciones arbitrarias del archivo cuando se requiere una operación más específica.
<code>ensure_file(path)</code>	Verifica que el archivo exista y, si no existe, lo crea.

El File Manager lleva contadores globales de lecturas y escrituras a disco. Esto permite que el Engine pueda reportar estadísticas de I/O por consulta. Todas las capas superiores, como Buffer Manager, Sequential File, B+ Tree, Extendible Hashing, R-Tree y los algoritmos externos, deben pasar por el File Manager para realizar operaciones físicas sobre memoria secundaria.

2.2.4. Buffer Manager

El Buffer Manager actúa como una capa de caché entre las estructuras de índice y el disco. Su objetivo es reducir el número de accesos físicos al almacenamiento mediante la retención en memoria de páginas recientemente utilizadas.

La implementación utiliza un pool de páginas con política de reemplazo LRU (Least Recently Used). Cada página se identifica mediante una clave compuesta por la ruta del archivo y el número de página. Cuando una estructura solicita una página, el Buffer Manager primero verifica si esta se encuentra en memoria. Si está presente, se produce un *buffer hit*; si no está, ocurre un *buffer miss* y la página debe ser leída desde disco.

Funcionamiento general

1. Una estructura solicita una página mediante `read_page()` o `write_page()`.
2. El Buffer Manager verifica si la página ya está cargada en el buffer pool.
3. Si la página está en memoria, se registra un hit y se reutiliza la página.
4. Si la página no está en memoria, se registra un miss y se lee desde disco usando File Manager.
5. Si el buffer está lleno, se expulsa la página menos recientemente utilizada.
6. Si la página expulsada está modificada, se escribe primero en disco mediante `flush()`.

Estadísticas recolectadas

Cuadro 6: Estadísticas del Buffer Manager

Métrica	Descripción
Hits	Cantidad de páginas encontradas directamente en memoria.
Misses	Cantidad de páginas que tuvieron que cargarse desde disco.
Disk reads	Lecturas físicas realizadas sobre archivos.
Disk writes	Escrituras físicas realizadas sobre archivos.
Flushes	Cantidad de veces que páginas modificadas fueron escritas a disco.

Además, el Buffer Manager integra mecanismos de concurrencia mediante bloqueos de lectura y escritura por página. Esto permite que múltiples lectores accedan simultáneamente a una misma página, pero garantiza exclusión mutua cuando una operación necesita escribir.

2.3. Estructuras de indexación sobre memoria secundaria

2.3.1. Sequential File

El Sequential File es una estructura de acceso a datos que organiza los registros en orden según una clave. La implementación se divide en dos archivos físicos: un archivo principal, que contiene los registros ordenados, y un archivo auxiliar, que almacena inserciones recientes sin ordenar.

El archivo principal permite realizar búsquedas binarias eficientes, mientras que el archivo auxiliar permite que las inserciones sean rápidas, ya que no se reordena todo el archivo en cada operación. Cuando el archivo auxiliar alcanza un límite máximo de registros, se ejecuta un proceso de reconstrucción o *rebuild*, que fusiona ambos archivos y vuelve a ordenar todos los registros activos.

Formato de registro

Cada registro ocupa 240 bytes fijos con el siguiente formato: key (float64, 8 bytes), datos (200 bytes), puntero al siguiente registro (int64, 8 bytes) y una bandera de eliminación (int32, 4 bytes). Cada página es de tamaño 4096.

Cada registro tiene tamaño fijo y contiene los siguientes campos:

Cuadro 7: Formato de registro en Sequential File

Campo	Tipo	Tamaño	Descripción
key	float64	8 bytes	Clave de búsqueda.
data	bytes	200 bytes	Información asociada, rellena con ceros si es necesario.
next_offset	int64	8 bytes	Puntero lógico al siguiente registro.
deleted	int32	4 bytes	Bandera de borrado lógico.

Inserción

La inserción se realiza agregando el nuevo registro al final del archivo auxiliar. Esto permite que la operación sea rápida, ya que no se modifica inmediatamente el archivo principal. Sin embargo, si el archivo auxiliar alcanza el límite definido, se ejecuta el proceso de `rebuild()`.

Búsqueda

La búsqueda se realiza en dos fases. Primero, se aplica búsqueda binaria sobre el archivo principal, aprovechando que este se encuentra ordenado. Luego, si la clave no se encuentra o puede existir en inserciones recientes, se revisa linealmente el archivo auxiliar.

Eliminación

La eliminación se maneja mediante borrado lógico. En lugar de eliminar físicamente el registro, se marca el campo `deleted` con valor 1. Esta decisión evita reorganizar el archivo en cada eliminación y mejora el rendimiento en operaciones de escritura.

Rebuild

Cuando el archivo auxiliar crece demasiado, el rendimiento de búsqueda disminuye. Por ello, el proceso de `rebuild()` carga todos los registros activos del archivo principal y auxiliar, los ordena por clave, sobrescribe el archivo principal y limpia el auxiliar.

Cuadro 8: Complejidad de Sequential File

Operación	Complejidad promedio	Observación
Inserción	$O(1)$	Se inserta al final del auxiliar.
Búsqueda	$O(\log n + k)$	Búsqueda binaria en principal y lineal en auxiliar.
Eliminación	$O(\log n + k)$	Se busca el registro y se marca como eliminado.
Rebuild	$O(N \log N)$	Fusiona y reordena registros activos.

2.3.2. Extendible Hashing

El Extendible Hashing es una estructura de indexación dinámica orientada a resolver consultas de búsqueda puntual por igualdad. Esta estructura no mantiene las claves ordenadas, sino que utiliza una función hash para ubicar directamente el bucket donde debería encontrarse una clave.

El índice es creado por el DBManager cuando una columna es declarada con índice de tipo hash. Internamente, el sistema traduce esa declaración a `IndexType.EXTHASH` y construye una instancia de `ExtendibleHash`.

El objetivo principal de Extendible Hashing es reducir el costo de una búsqueda exacta. En condiciones normales, esto permite un costo promedio cercano a $O(1)$. Sin embargo, este costo depende de que las claves estén distribuidas adecuadamente y de que las cadenas de overflow no crezcan demasiado.

Modelo de almacenamiento en disco

Se utilizan dos archivos físicos principales: `index.dat` y `data.dat`. Esta organización es conveniente porque el directorio puede crecer sin necesidad de mover todos los buckets físicos, y los buckets pueden almacenarse en páginas de tamaño fijo.

Cuadro 9: Archivos físicos de Extendible Hashing

Archivo	Descripción
index.dat	Guarda el encabezado del índice y el directorio global.
data.dat	Guarda los buckets donde se almacenan claves y valores asociados.

Formato de index.dat

El archivo `index.dat` almacena la metadata necesaria para reconstruir el estado del índice.

Cuadro 10: Encabezado de index.dat

Campo	Descripción
global_depth	Cantidad de bits utilizados para indexar el directorio global.
directory_size	Número de entradas del directorio. Equivale a 2^{global_depth} .
free_head	Apunta al primer bucket libre disponible para re-utilización.
bucket_count	Cantidad total de buckets creados.
fb	Factor de bucket, es decir, número máximo de claves que puede almacenar cada bucket.

El directorio global es un arreglo de referencias a buckets. Cada entrada no almacena el bucket completo, sino una referencia física identificada por `page_id` y `slot_id`. Esto permite ubicar el bucket correspondiente dentro de `data.dat`.

Para determinar qué entrada del directorio corresponde a una clave, se utilizan los bits menos significativos:

$$\text{dir_idx} = \text{key} \& ((1 \ll \text{global_depth}) - 1)$$

La cantidad de bits usados depende de `global_depth`. Varios índices del directorio pueden apuntar al mismo bucket cuando la profundidad local del bucket es menor que la profundidad global.

Formato de `data.dat`

El archivo `data.dat` almacena los buckets. Cada bucket contiene un número fijo de pares clave-valor determinado por `fb`.

Cuadro 11: Estructura de un bucket en Extendible Hashing

Atributo	Descripción
<code>local_depth</code>	Número de bits que distinguen al bucket.
<code>count</code>	Cantidad de entradas ocupadas.
<code>keys</code>	Arreglo de claves almacenadas.
<code>values</code>	Arreglo de valores asociados, normalmente <code>record_id</code> .
<code>next</code>	Puntero al bucket de overflow.
<code>free_list</code>	Puntero utilizado para reutilizar buckets libres.

Los buckets se almacenan dentro de páginas de 4096 bytes. Como una página puede contener varios buckets, se calcula la página y el slot exacto:

```
page_id = bucket_id // buckets_per_page
slot_id = bucket_id % buckets_per_page
```

Esto permite aprovechar mejor cada página de disco y mantener acceso directo a los buckets sin recorrer secuencialmente el archivo.

Inserción

La inserción sigue el siguiente procedimiento:

1. Se lee el encabezado del índice y el directorio global.
2. Se calcula el índice del directorio usando la clave.
3. Se obtiene el bucket apuntado por esa entrada.
4. Se verifica si la clave ya existe.
5. Si el bucket tiene espacio, se inserta el par (`key`, `value`).
6. Si el bucket está lleno, se ejecuta un proceso de split.
7. Después del split, se reintenta la inserción.

El split permite que el índice crezca dinámicamente sin reconstruir toda la estructura.

Cuadro 12: Casos de split en Extendible Hashing

Caso	Acción
<code>local_depth < global_depth</code>	El directorio ya tiene suficiente profundidad, por lo que solo se crea un nuevo bucket y se redistribuyen las claves.
<code>local_depth == global_depth</code>	El directorio no tiene suficientes bits para diferenciar el nuevo bucket, por lo que se duplica el directorio y luego se redistribuyen las claves.
<code>global_depth == MAX_GLOBAL_DEPTH</code>	Ya no se puede duplicar el directorio, por lo que se conectan buckets de overflow.

En los tres casos, primero se verifica si existen buckets libres mediante `free_head` antes de crear un nuevo bucket.

Búsqueda

La búsqueda se realiza de la siguiente manera:

1. Leer `global_depth`.
2. Calcular el índice del directorio.
3. Acceder al bucket correspondiente.
4. Buscar la clave dentro del bucket.
5. Si no se encuentra y existe overflow, recorrer la cadena enlazada.
6. Retornar el valor asociado o `None`.

Eliminación

La eliminación busca la clave dentro del bucket correspondiente. Si la clave se encuentra, se remueve del arreglo de claves y valores, y se reduce el contador `count`. Si la clave se encuentra dentro de un bucket de overflow y dicho bucket queda vacío, este puede desconectarse de la cadena y agregarse a la lista libre mediante `free_head`.

La implementación no realiza merge de buckets ni reducción del directorio después de eliminar. Esta decisión simplifica la estructura y evita reorganizaciones costosas, aunque puede dejar un directorio más grande que el estrictamente necesario.

Cuadro 13: Complejidad de Extendible Hashing

Operación	Complejidad promedio
Inserción sin split	$O(1)$
Inserción con split	$O(fb + local_depth)$
Búsqueda	$O(1 + longitud\ de\ overflow)$
Eliminación	$O(1)$

Limitaciones

- No soporta búsqueda de rangos.
- Puede degradarse si muchas claves caen en cadenas de overflow.
- Trabaja mejor con columnas de claves únicas, como primary keys.

2.3.3. B+ Tree

El B+ Tree es una estructura de indexación ampliamente utilizada en sistemas de gestión de bases de datos debido a su eficiencia en operaciones de búsqueda, inserción y eliminación sobre grandes volúmenes de datos almacenados en memoria secundaria.

En esta implementación, el B+ Tree ha sido diseñado específicamente para operar sobre archivos binarios, organizando la información en páginas de tamaño fijo. Esto permite optimizar el acceso a disco mediante operaciones de lectura y escritura alineadas con bloques.

El objetivo principal de esta estructura es minimizar el número de accesos a disco (I/O), manteniendo el árbol balanceado y garantizando complejidad logarítmica en las operaciones.

Modelo de almacenamiento en disco

El índice se almacena en un archivo `.bpt`, dividido en páginas de tamaño fijo:

`PAGE_SIZE = 4096`

Cada página representa un nodo del árbol. El acceso directo a una página se realiza mediante:

```
f.seek(pid * PAGE_SIZE)
```

Estructura de una página

Cada página está compuesta por un `[HEADER | BODY]`. El header de página está definido en el método `serialize()` por:

Cuadro 14: Header de página en B+ Tree

Campo	Tamaño	Descripción
B	1 byte	Indica si el nodo es hoja (1) o interno (0).
h	2 bytes	Número de claves almacenadas en la página.
q	8 bytes	Puntero a la siguiente hoja (<code>next_leaf</code>), por defecto -1.

Representación de nodos

Cada página del archivo es representada en memoria mediante la clase `Node`, la cual encapsula la estructura lógica de los nodos del árbol B+.

Nodo hoja: los nodos hoja contienen los datos reales del índice, almacenando pares clave-valor:

- **keys:** lista de claves ordenadas.
- **values:** datos asociados a cada clave.
- **next_leaf:** puntero a la siguiente hoja.

La serialización de estos nodos se realiza en el método `serialize()`, donde cada clave es empaquetada junto a su valor correspondiente, con `DATA_SIZE = 200 bytes`.

La existencia del puntero `next_leaf` permite enlazar secuencialmente todas las hojas del árbol, facilitando la ejecución de consultas por rango.

Nodo interno: los nodos internos contienen únicamente claves de separación y punteros a nodos hijos:

- **keys:** claves que dividen el espacio de búsqueda.
- **children:** punteros a páginas hijas.

En el método `serialize()`, estos nodos se representan alternando punteros y claves, comenzando con un puntero inicial. Su función principal es guiar el proceso de búsqueda hacia la hoja adecuada.

Serialización y deserialización

Serialización: el método `serialize()` convierte un nodo en una secuencia de bytes para su almacenamiento en disco. Este proceso incluye:

- empaquetado del header;
- codificación de claves según su tipo (`int`, `float`, `str`);
- relleno de la página hasta alcanzar el tamaño fijo `PAGE_SIZE`.

Deserialización: el método `from_bytes()`, implementado como `@classmethod`, permite reconstruir un nodo a partir de los bytes leídos desde el archivo. Este método interpreta el header y reconstruye las listas de claves, valores o hijos dependiendo del tipo de nodo.

Gestión de páginas

La interacción con el almacenamiento en disco se realiza mediante tres operaciones principales:

- **Lectura de página:** implementada en el método `_read`, donde se utiliza `seek` para posicionarse en el offset correspondiente y se leen `PAGE_SIZE` bytes.
- **Escritura de página:** implementada en `_write`, sobrescribiendo el contenido de una página existente en disco.

- **Creación de nuevas páginas:** el método `_new_page` agrega nuevas páginas al final del archivo, calculando el nuevo `page_id` mediante la posición actual del archivo con `f.tell()`.

Propiedades del B+ Tree

La implementación cumple con las propiedades fundamentales del B+ Tree:

- árbol completamente balanceado;
- todas las hojas se encuentran al mismo nivel;
- claves almacenadas en orden;
- complejidad de búsqueda $O(\log n)$;
- optimización para acceso secuencial mediante hojas enlazadas.

Operaciones principales

Búsqueda: el método `search()` localiza una clave siguiendo un recorrido descendente desde la raíz hasta una hoja, utilizando el método `_find_leaf`. En la hoja, la búsqueda se realiza de forma secuencial.

Inserción: el método `add()` inserta una nueva clave en el árbol:

1. Se localiza la hoja correspondiente.
2. Se inserta la clave en orden mediante `_insert_leaf`.
3. Si la página excede su capacidad, se ejecuta un split.

Split de nodos: cuando un nodo supera su capacidad máxima, se realiza una división.

Split en hoja (`_split_leaf`):

- se divide el nodo en dos;
- se actualizan los punteros `next_leaf`;
- se propaga la primera clave del nuevo nodo al padre.

Split en nodo interno (`_split_internal`):

- se divide el nodo;
- se promueve la clave central al nivel superior.

Eliminación: el método `remove()` gestiona la eliminación de claves, incluyendo:

- eliminación directa en nodos hoja;
- redistribución de claves entre nodos hermanos;
- fusión de nodos en caso de underflow.

Estas operaciones aseguran que el árbol mantenga sus propiedades estructurales.

Búsqueda por rango: el método `range_search()` permite recuperar valores dentro de un intervalo de claves. Esta operación se optimiza gracias al enlace entre nodos hoja (`next_leaf`), permitiendo un recorrido secuencial eficiente.

Cálculo de capacidad de nodos

La capacidad máxima de cada nodo se calcula en el método `_setup_key_format`.

Para nodos hoja:

```
_leaf_max = (PAGE_SIZE - HEADER_SIZE) // (key_size + DATA_SIZE)
```

Para nodos internos:

```
_internal_max = (PAGE_SIZE - HEADER_SIZE - PTR_SIZE) // (key_size + PTR_SIZE)
```

Estos valores determinan el número máximo de entradas por página.

Persistencia del índice

El sistema utiliza dos archivos:

- `.bpt`: almacena las páginas del árbol.
- `.meta`: almacena información de la raíz y el tipo de clave.

Los métodos `_save_meta` y `_load_meta` permiten conservar el estado del árbol entre ejecuciones.

Consideraciones de diseño

Las principales decisiones de diseño incluyen:

- uso de páginas de tamaño fijo para optimizar el acceso a disco;
- separación clara entre nodos internos y hojas;
- implementación de enlaces entre hojas para consultas por rango;
- uso de representación binaria eficiente mediante la librería `struct`;
- soporte para múltiples tipos de clave.

2.3.4. R-Tree

Para indexar eficientemente los registros geográficos del proyecto se implementó un R-Tree, una estructura de datos espacial diseñada para manejar objetos en dos o más dimensiones. A diferencia de un árbol B tradicional, que organiza claves unidimensionales, el R-Tree agrupa objetos en el plano mediante MBR (Minimum Bounding Rectangle), es decir, el rectángulo mínimo que encierra un conjunto de puntos.

En este proyecto cada registro representa una coordenada geográfica definida por longitud y latitud, por lo que el índice debe responder eficientemente consultas espaciales como:

- **Range Search:** determinar qué puntos se encuentran dentro de un radio respecto a un centro (c_x, c_y) .
- **K-Nearest Neighbors (k-NN):** encontrar los k puntos más cercanos a una coordenada dada.

Si estas consultas se resolvieran secuencialmente, sería necesario recorrer todos los registros, con costo $O(n)$. El R-Tree permite podar ramas completas mediante los MBR, reduciendo el costo promedio aproximadamente a $O(\log n)$.

Concepto clave: MBR (Minimum Bounding Rectangle)

Un MBR es el rectángulo más pequeño que contiene a todos los puntos de un nodo.

Si una consulta, por ejemplo un círculo de búsqueda, NO intersecta el MBR de un nodo, se descartan TODOS los hijos de ese nodo sin leerlos. Esto es la poda del árbol.

Constantes y configuración de página

Lo primero que hace el archivo es definir las constantes que rigen el tamaño de las páginas en disco y el layout binario de cada entrada:

```
PAGE_SIZE = 4096 # Size de pg = 4 KB (igual que un bloque de disco)
DATA_SIZE = 160 # Bytes reservados para el payload de cada punto
MBR_FMT = '4d' # 4 doubles: x_min, y_min, x_max, y_max
MBR_SIZE = 32 bytes # struct.calcsize('4d') = 4 * 8 = 32
CHILD_SIZE = 8 bytes # struct.calcsize('q') = int64
ENTRY_SIZE = MBR_SIZE + max(CHILD_SIZE, DATA_SIZE) = 192 bytes
HEADER_FMT = '<Bh' # is_leaf (u8) + n_entries (i16) = 3 bytes
M = (4096 - 3) // 192 = 21 # Max de entradas por nodo
m = max(1, M // 2) = 10 # Min de entradas por nodo (llenado min)
```

Fórmula de M y m

- **M:** máximo de entradas por nodo.

$$M = (\text{PAGE_SIZE} - \text{HEADER_SIZE}) / \text{ENTRY_SIZE}$$

- **m:** mínimo de entradas por nodo.

$$m = M / 2$$

Esta relación garantiza que el árbol nunca quede vacío en sus nodos internos, manteniendo el balance y la eficiencia de las operaciones de búsqueda.

El Rectángulo Mínimo Delimitador

La clase **Rect** es la pieza fundamental de todo el R-Tree. Representa un MBR definido por cuatro coordenadas flotantes:

```
class Rect:
    def __init__(self, x_min, y_min, x_max, y_max):
        self.x_min = x_min
        self.y_min = y_min
        self.x_max = x_max
        self.y_max = y_max
```

Métodos de Rect

Cuadro 15: Métodos de la clase Rect

Método	¿Qué hace?	¿Para qué se usa?
from_point(x,y)	Crea un MBR degenerado.	Inserción de nuevos puntos geográficos.
area()	Calcula el área del rectángulo.	Elegir subárboles en inserción.
enlargement(other)	Calcula cuánto crecería el área.	Selección de subárbol.
merge(other)	Une dos rectángulos.	Recalcular MBRs.
intersects(other)	Verifica intersección.	Poda en búsquedas.
intersects_circle(cx,cy,r)	Verifica intersección con círculo.	Range Search.
min_dist(cx,cy)	Distancia mínima al rectángulo.	KNN.

El método **intersects_circle()** merece atención especial. En lugar de comprobar directamente si el círculo y el rectángulo se solapan, calcula el punto más cercano del rectángulo al centro del círculo. Si la distancia de ese punto al centro es menor o igual al radio, entonces el círculo intersecta el rectángulo.

Entry, Leaf y Branch — Tipos de entradas

El árbol maneja dos tipos de entradas que heredan de **Entry**:

```
class Entry:
    def __init__(self, rect: Rect):
        self.rect = rect

class Leaf(Entry):
    def __init__(self, rect, data: bytes):
        self.data = data
```

```
class Branch(Entry):
    def __init__(self, rect, child_page: int):
        self.child_page = child_page
```

Diseño clave: datos en hojas, punteros en nodos internos

- Los nodos hoja contienen entradas `Leaf` con el dato real.
- Los nodos internos contienen entradas `Branch` con el `page_id` del hijo.

Este es el diseño clásico de R-Tree: la data solo vive en el nivel hoja. Los nodos internos únicamente guían la búsqueda.

Nodo en disco

`serialize()` — Serializar a bytes

Este método convierte un nodo en exactamente `PAGE_SIZE` bytes para escribirlos en disco.

El proceso incluye:

- empaquetado del header;
- serialización de cada MBR;
- escritura del payload;
- relleno con ceros hasta completar 4096 bytes.

`from_bytes()` — Deserializar desde bytes

Reconstruye un nodo leyendo el header y luego interpretando cada entrada utilizando offsets calculados con `ENTRY_SIZE`.

```
off = HEADER_SIZE + i * ENTRY_SIZE
```

Clase `RTree` — El motor

`__init__()` — Inicialización

El constructor gestiona la persistencia del árbol:

- Si el archivo `.rtree` no existe, crea la raíz vacía.
- Si el archivo ya existe, carga la metadata del árbol.

El archivo `.rtree.meta` almacena:

- `root_page`
- `page_count`

Esto permite retomar el árbol exactamente donde terminó la ejecución anterior.

`_lock_and_pin()` — Gestión de Buffer Manager

Este context manager encapsula cualquier acceso a página junto con operaciones de locking del Buffer Manager. Cuando existe Buffer Manager, realiza:

- `lock_page()`
- lectura/escritura
- `unlock()`
- `unpin()`

Cuando el árbol trabaja standalone, simplemente ejecuta el bloque normalmente.

Inserción — `add()` e `_insert()`

La inserción sigue el algoritmo clásico de Guttman (1984).

`add(x, y, data)`

- crea un `Rect.from_point()`;
- crea un objeto `Leaf`;
- llama a `_insert()` desde la raíz.

`_insert(page_id, entry)`

- si el nodo es hoja, añade directamente la entrada;
- si hay overflow, divide el nodo;
- si el nodo es interno, elige el subárbol adecuado.

`_choose_subtree()`

Selecciona el hijo cuyo MBR crece menos al insertar la nueva entrada utilizando el criterio mínimo `enlargement()`.

Propagación

Si un hijo se divide, el nodo padre recibe dos nuevas ramas. Si el padre excede su capacidad, también se divide.

Nueva raíz

Si la raíz se divide, se crea una nueva raíz con dos entradas `Branch` apuntando a las dos mitades. El árbol crece en altura por arriba.

`_pick_seeds()` — Elegir semillas

Cuando un nodo supera M entradas, se utiliza Quadratic Split. El método `_pick_seeds()` elige el par de entradas que produciría mayor desperdicio de área si estuvieran juntas.

`_pick_next()` — Asignar siguiente entrada

Para cada entrada pendiente calcula cuánto crecería cada grupo. La entrada se asigna al grupo que menos crezca.

`_split()` — Proceso completo

- se eligen semillas;
- se crean dos grupos;
- se redistribuyen las entradas;
- el nodo original reutiliza su página;
- el nuevo grupo se escribe en una nueva página.

Búsqueda por rango — `range_search()`

Esta función responde:

”Dame todos los puntos dentro del círculo de radio r centrado en (c_x, c_y) ”.

Fase 1 — Filtro aproximado con MBR

Para cada entrada del nodo:

- si el MBR NO intersecta el círculo, se descarta toda la rama;
- esto evita leer páginas innecesarias.

Fase 2 — Verificación exacta

En hojas, cada punto candidato se verifica utilizando distancia euclidiana real:

```
math.hypot(cx - x, cy - y) <= r
```

Solo los puntos válidos se agregan al resultado.

K vecinos más cercanos — `knn()`

Implementa Best-First Search utilizando un heap de prioridad (`heapq`).

La idea principal es expandir primero los nodos más prometedores, es decir, aquellos cuyo MBR está más cerca del punto de consulta.

¿Por qué Best-First Search?

La alternativa naive sería ejecutar múltiples range searches aumentando el radio. Sin embargo, esto es ineficiente e impredecible.

Best-First Search utiliza:

- distancia exacta para puntos hoja;
- distancia mínima al MBR para nodos internos.

Cuando el heap extrae un punto real como el mínimo, se garantiza que no existe otro punto más cercano sin procesar.

Paso a paso del kNN

1. El heap comienza con la raíz.
2. Se expande el nodo más cercano.
3. Se insertan hijos o puntos candidatos.
4. Cuando se extrae un punto real, se confirma como vecino.
5. Se repite hasta obtener k resultados.

Eliminación — `remove()`

La eliminación en R-Trees es más compleja que en B-Trees porque requiere reorganización. El algoritmo utilizado es *Condense Tree* de Guttman.

1. `_remove_recursive()` localiza el punto.
2. Si un nodo queda con menos de m entradas, se elimina.
3. Sus entradas se agregan a una lista temporal.
4. `_reinsert_subtree()` reinserta dichas entradas.
5. Si la raíz queda con un solo hijo, el árbol reduce su altura.

Propiedades del R-Tree

- árbol balanceado;
- optimización espacial mediante MBRs;
- reducción de accesos a disco;
- soporte para búsquedas espaciales;
- soporte para consultas KNN.

Figura 2: Referencia para diagrama de R-Tree

2.4. Parser SQL personalizado

El parser SQL está compuesto por dos módulos principales: el scanner y el parser propiamente dicho. Esta separación sigue la arquitectura clásica de compiladores, donde primero se realiza análisis léxico y luego análisis sintáctico.

2.4.1. Gramática

La gramática está definida formalmente en notación EBNF. El sistema soporta sentencias de creación de tablas, inserción, eliminación, selección, filtros, joins, ordenamiento, agrupación y consultas espaciales.

```
program ::= { statement ';' }
```



```
statement ::=
    'CREATE' 'TABLE' IDENTIFIER
        '(' column_def { ',' column_def } ')'
    [ 'FROM' 'FILE' PATH ]
| 'SELECT' '*' 'FROM' IDENTIFIER
    [ 'JOIN' IDENTIFIER 'ON' column_ref '=' column_ref ]
    [ 'WHERE' condition ]
    [ 'ORDER' 'BY' IDENTIFIER [ 'ASC' | 'DESC' ] ]
| 'INSERT' 'INTO' IDENTIFIER 'VALUES' '(' value { ',' value } ')'
| 'DELETE' 'FROM' IDENTIFIER 'WHERE' IDENTIFIER '=' value
```



```
column_def ::=
    IDENTIFIER data_type [ 'INDEX' index_type ]
```



```
data_type ::=
    'INT' | 'FLOAT' | 'VARCHAR' '(' INT ')' | 'BOOL' | 'POINT'
```



```
index_type ::=
    'SEQUENTIAL' | 'HASH' | 'BTREE' | 'RTREE'
```



```
condition ::=
    IDENTIFIER '=' value
| IDENTIFIER 'BETWEEN' value 'AND' value
| IDENTIFIER 'IN' '(' point ',' 'RADIUS' NUMBER ')'
| IDENTIFIER 'IN' '(' point ',' 'K' INT ')'
```



```
column_ref ::=
    IDENTIFIER [ '.' IDENTIFIER ]
```



```
value ::=
    INT | FLOAT | STRING | BOOLEAN | point
```



```
point ::=
```

`'POINT' '(' NUMBER ',' NUMBER ')'`

IDENTIFIER ::= LETTER { LETTER | DIGIT | '_' }
INT ::= ['-'] DIGIT { DIGIT }
FLOAT ::= ['-'] DIGIT { DIGIT } '.' DIGIT { DIGIT }
NUMBER ::= INT | FLOAT
STRING ::= " " { CHAR } "
BOOLEAN ::= 'TRUE' | 'FALSE'
PATH ::= " " { CHAR } "
LETTER ::= 'a' | ... | 'z' | 'A' | ... | 'Z'
DIGIT ::= '0' | ... | '9'
CHAR ::= cualquier car cter excepto " "

2.4.2. Scanner

El Scanner realiza el análisis léxico del texto SQL. Reconoce palabras clave, tipos de datos, tipos de índice, literales numéricos, strings, booleanos, símbolos e identificadores.

Entre los tokens reconocidos se encuentran:

- SELECT, INSERT, CREATE, DELETE
- FROM, WHERE, JOIN, ON
- BETWEEN, RADIUS, K
- GROUP, BY, COUNT
- ORDER, ASC, DESC
- INT, FLOAT, VARCHAR, BOOL, POINT
- HASH, BTREE, RTREE, SEQUENTIAL

El reconocimiento de palabras clave es insensible a mayúsculas. Además, el scanner reporta errores indicando línea y columna, lo cual facilita la depuración de consultas incorrectas.

2.4.3. Parser

El Parser implementa un parser de descenso recursivo. Este recibe la lista de tokens generada por el Scanner y construye un AST (Abstract Syntax Tree), que representa la consulta en una estructura procesable por el Engine.

El parser soporta:

- SELECT simple
- SELECT con WHERE
- SELECT con JOIN

- SELECT con GROUP BY y COUNT(*)
- SELECT con ORDER BY
- CREATE TABLE
- CREATE TABLE FROM FILE
- INSERT INTO
- DELETE FROM
- búsquedas espaciales por radio
- búsquedas espaciales KNN

Ejemplo de flujo

```
SELECT categoria , COUNT(*)
FROM ventas
GROUP BY categoria ;
```

El Scanner genera una secuencia de tokens:

```
[SELECT, IDENTIFIER('categoria'), COMMA, COUNT,
 LPAREN, STAR, RPAREN, FROM, IDENTIFIER('ventas'),
 GROUP, BY, IDENTIFIER('categoria'), SEMICOLON, EOF]
```

Luego, el Parser construye un nodo AST de tipo SELECT con las columnas seleccionadas y el campo `group_by`.

Finalmente, el Engine ejecuta la consulta, recupera las filas mediante DBManager y aplica la agrupación correspondiente.

Autómata del Scanner

Cuadro 16: Autómata del Scanner en formato de tabla

Estado actual	Input	Siguiente estado	Acción / Token emitido
INICIO	espacio / tab / \n	INICIO	skip
INICIO	--	COMENTARIO	skip hasta \n
INICIO	() , ; = *	INICIO	emitir símbolo correspondiente
INICIO	'	STRING	avanzar
INICIO	dígito o - seguido de dígito	NUMERO	acumular
INICIO	letra o _	IDENT	acumular
INICIO	EOF	—	emitir EOF
STRING	cualquier char $\neq$ '	STRING	acumular en buffer
STRING	'	INICIO	emitir STRING(buffer)
STRING	EOF	ERROR	“cadena sin cerrar”

Estado actual	Input	Siguiente estado	Acción / Token emitido
NUMERO	dígito	NUMERO	acumular
NUMERO	. seguido de dígito	NUMERO.FLOAT	acumular
NUMERO	otro	INICIO	emitir INT(buffer)
NUMERO.FLOAT	dígito	NUMERO.FLOAT	acumular
NUMERO.FLOAT	otro	INICIO	emitir FLOAT(buffer)
IDENT	letra / dígito / _	IDENT	acumular
IDENT	otro	INICIO	lookup en tabla KEYWORDS → emitir keyword o IDENTIFIER
COMENTARIO	char $\neq$ \n	COMENTARIO	skip
COMENTARIO	\n / EOF	INICIO	—

3. Consideraciones técnicas

Durante el desarrollo del proyecto se priorizó mantener una arquitectura modular similar a la utilizada por sistemas gestores de bases de datos reales. Todas las estructuras fueron implementadas sobre páginas de tamaño fijo de 4096 bytes, permitiendo uniformidad entre índices, archivos de registros y algoritmos externos.

El sistema fue diseñado para trabajar directamente sobre memoria secundaria, minimizando accesos innecesarios a disco mediante el uso de un Buffer Manager con política LRU. Asimismo, se optó por utilizar serialización binaria manual mediante struct y pack/unpack para reducir overhead y mantener control explícito sobre el layout físico de las estructuras.

Para las búsquedas espaciales se utilizó un R-Tree basado en MBRs, mientras que para búsquedas exactas se implementaron estructuras como Extendible Hashing y B+ Tree. Adicionalmente, se incorporaron algoritmos de memoria externa para operaciones de ordenamiento y joins, permitiendo trabajar con datasets de mayor tamaño sin depender completamente de memoria RAM.

4. Simulador de acceso concurrente

4.1. Control de concurrencia: decisiones de implementación

Para cumplir con el componente de control de concurrencia del proyecto y obtener los puntos adicionales, se implementaron tres mecanismos que trabajan en conjunto como un sistema unificado. :contentReference[oaicite:0]index=0

Bloqueos compartidos/exclusivos a nivel de página (Shared/Exclusive Locks)

Se implementó una clase `RWLock` que diferencia entre dos tipos de acceso a cada página en disco: acceso compartido para lecturas simultáneas y acceso exclusivo para escrituras. :contentReference[oaicite:1]index=1

Múltiples hilos pueden leer una página al mismo tiempo, pero en el momento en que un hilo necesita escribir, obtiene acceso exclusivo y bloquea a todos los demás.

Grafo de espera con detección de ciclos (Wait-For Graph + DFS)

Se implementó un `DeadlockDetector` que mantiene en tiempo real un grafo dirigido de dependencias entre transacciones.

Cada vez que una transacción espera por una página que otra transacción tiene bloqueada, se registra esa relación en el grafo. Sobre ese grafo se ejecuta un algoritmo DFS que busca ciclos. Si encuentra uno, significa que hay un deadlock real y lanza inmediatamente una excepción controlada.

Log causal de operaciones

Se mejoró el `TransactionLogger` para registrar no solo que ocurrió un evento, sino el estado exacto de cada lock en el momento en que ocurrió, mostrando la cadena causal completa: quién pidió qué, quién esperó a quién, quién fue abortado y quién completó.

4.2. Justificación de las decisiones de diseño

La razón central de elegir esta arquitectura es que el enunciado del proyecto exige dos cosas distintas: control de concurrencia y detección de interbloqueos. Muchas implementaciones simples confunden ambas o resuelven solo una. A continuación se explica por qué cada decisión fue la correcta.

¿Por qué RWLock y no un Lock simple?

Un `threading.Lock()` o `threading.RLock()` estándar de Python no distingue entre lectura y escritura. Esto significa que dos hilos haciendo `SELECT` simultáneamente se bloquean mutuamente aunque no exista ningún conflicto real entre ellos.

Con `RWLock`, múltiples lectores conviven sin bloquearse, y solo cuando existe una escritura se impone exclusividad. Esto replica el comportamiento real de un gestor de base de datos y permite mayor paralelismo en operaciones de consulta.

¿Por qué Wait-For Graph con DFS y no solo Timeout?

Un timeout simplemente aborta una transacción cuando espera demasiado tiempo, sin saber si realmente existe un deadlock o si el lock se hubiera liberado momentos después. Esto genera abortos innecesarios y no demuestra comprensión del problema.

El Wait-For Graph, en cambio, detecta el deadlock en el instante exacto en que se forma el ciclo, sin esperar ningún tiempo arbitrario. Cuando TX-A espera a TX-B y TX-B espera a TX-A, el DFS recorre el grafo, encuentra el ciclo y lanza `DeadlockDetectedError` de inmediato.

Este es el mismo principio utilizado internamente por gestores de bases de datos reales como PostgreSQL e InnoDB.

¿Por qué el patrón Singleton en DeadlockDetector?

El grafo de espera debe ser único y global. Si cada hilo o cada índice tuviera su propia instancia del detector, cada uno vería solo una parte del grafo y nunca detectaría ciclos entre transacciones que operan sobre distintos índices.

El patrón Singleton garantiza que todos los hilos, sin importar qué índice estén utilizando, alimenten y consulten el mismo grafo compartido.

¿Por qué integrar los locks en el BufferManager y no en cada índice individualmente?

Integrar el control de concurrencia directamente en cada índice habría generado código duplicado y habría dejado sin protección las operaciones del archivo base de registros.

Al centralizar el control en el BufferManager, cualquier estructura que necesite leer o escribir una página en disco pasa obligatoriamente por el mismo punto de control. Esto garantiza cobertura total sin importar desde qué índice se origina la operación.

4.3. Archivos modificados y creados

4.3.1. `indexes/deadlock_detector.py`

Este archivo no existía en el repositorio original. Se creó para implementar el grafo de espera y el algoritmo DFS. Contiene la clase `DeadlockDetector` como Singleton y la excepción `DeadlockDetectedError`.

Se colocó en la carpeta `indexes/` porque es una dependencia directa del BufferManager, que también vive ahí, y porque su función pertenece a la infraestructura de acceso a datos y no a lógica de negocio.

4.3.2. `indexes/rw_lock.py`

Este archivo tampoco existía originalmente. Se creó para implementar la clase `RWLock` utilizando `threading.Condition`, que es la primitiva adecuada de Python para coordinar múltiples lectores y un escritor.

Al igual que el detector de deadlocks, se colocó en `indexes/` por ser una dependencia directa del BufferManager.

4.3.3. `indexes/buffer_manager.py`

Este archivo ya existía y era el componente central de gestión de páginas, pero originalmente utilizaba `threading.RLock()` sin distinguir tipos de acceso ni detectar deadlocks.

Se modificó para:

- importar `RWLock` y `DeadlockDetector`;
- reemplazar locks simples por `RWLock` en el diccionario de locks por página;

- reescribir `lock_page()` y `unlock_page()` para:
 - registrar espera en el grafo;
 - verificar ciclos;
 - adquirir locks compartidos o exclusivos;
 - registrar el resultado en el log causal.

4.3.4. `manager/record_file.py`

Este archivo administra el archivo físico de registros base, independientemente de los índices. Sin modificarlo, los índices estarían protegidos pero el archivo de datos subyacente no, dejando una brecha real en el sistema de concurrencia.

Se modificó para recibir el `BufferManager` como parámetro opcional y agregar llamadas a `lock_page()`:

- con `lock_type="shared"` en operaciones de lectura (`read_slot`, `scan_all`);
- con `lock_type="exclusive"` en operaciones de escritura (`add`, `delete`).

4.3.5. `manager/transaction_log.py`

El logger original registraba eventos simples sin mostrar relaciones causales entre ellos. Se modificó para agregar el método `log_causal()`, que registra cada evento con su estado explícito en el formato:

```
[TX-ID] LOCK TIPO p gina N ESTADO (detalle)
```

Esto permite reconstruir exactamente qué ocurrió, en qué orden y por qué una transacción fue abortada o completada.

4.3.6. `manager/db_manager.py`

Se modificó para pasar el `BufferManager` al construir cada `RecordFile`, cerrando así el ciclo de integración. Sin este cambio, `RecordFile` nunca hubiera recibido la referencia al `BufferManager` y los locks definidos en `record_file.py` nunca se habrían activado.

4.3.7. `simulador_concurrencia.py`

Script ejecutable ubicado en la raíz del proyecto que demuestra funcionalmente todo el sistema de concurrencia.

Contiene cinco escenarios:

- el deadlock clásico cruzado entre TX-A y TX-B;
- cuatro escenarios de estrés concurrente, uno por cada índice:
 - Sequential File
 - B+ Tree
 - Extendible Hashing
 - R-Tree

Cada escenario utiliza `threading.Barrier` para garantizar que los hilos de lectura y escritura compitan simultáneamente por las mismas páginas.

Cada ejecución finaliza con una verificación de consistencia post-ejecución.

5. Interfaz gráfica de usuario

La interfaz gráfica es una aplicación web construida con React y Vite, la cual se comunica con el backend mediante una API REST desarrollada en FastAPI.

El objetivo de la interfaz fue permitir que el usuario pueda ejecutar consultas SQL de manera interactiva y visualizar tanto los resultados como las estadísticas del motor.

5.1. Componentes principales

Cuadro 17: Componentes de la interfaz gráfica

Componente	Descripción
Editor SQL	Área de texto para escribir consultas SQL. Incluye resaltado de sintaxis.
Panel de resultados	Muestra las filas retornadas por la consulta en formato tabular.
Panel de estadísticas	Muestra lecturas, escrituras, hits, misses, uso del buffer y tiempo de ejecución.
Visualización espacial	Permite mostrar puntos geográficos en un mapa interactivo mediante Leaflet.
Badges dinámicos	Indican si se usaron algoritmos como External Sort, Replacement Selection o External Hashing.

5.2. Funcionalidades

La interfaz permite:

- crear tablas desde SQL;
- cargar datasets desde archivo;

- insertar registros;
- ejecutar búsquedas por igualdad;
- ejecutar búsquedas por rango;
- ejecutar consultas espaciales;
- ejecutar joins;
- ordenar resultados;
- visualizar estadísticas de memoria secundaria.

6. Evaluación experimental

6.1. Metodología

Para la evaluación experimental se utilizó un dataset real de Airbnb obtenido de Kaggle, el cual contiene información de propiedades como precio, identificador y ubicación geográfica.

Se generaron subconjuntos de tamaños:

- $n = 1,000$
- $n = 10,000$
- $n = 100,000$

Las estructuras evaluadas fueron:

- B+ Tree
- Extendible Hashing
- Sequential File
- R-Tree

Las métricas utilizadas fueron:

Cuadro 18: Métricas experimentales

Métrica	Descripción	Unidad
Accesos a disco	Lecturas y escrituras	Páginas
Tiempo de ejecución	Tiempo total	ms

6.2. Resultados experimentales

6.2.1. Inserción

Sequential File

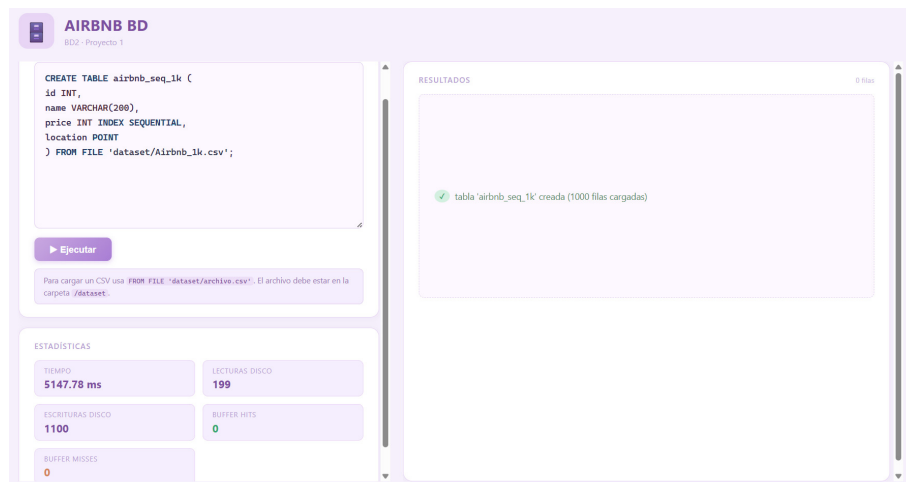


Figura 3: Inserción – Sequential File – 1k registros

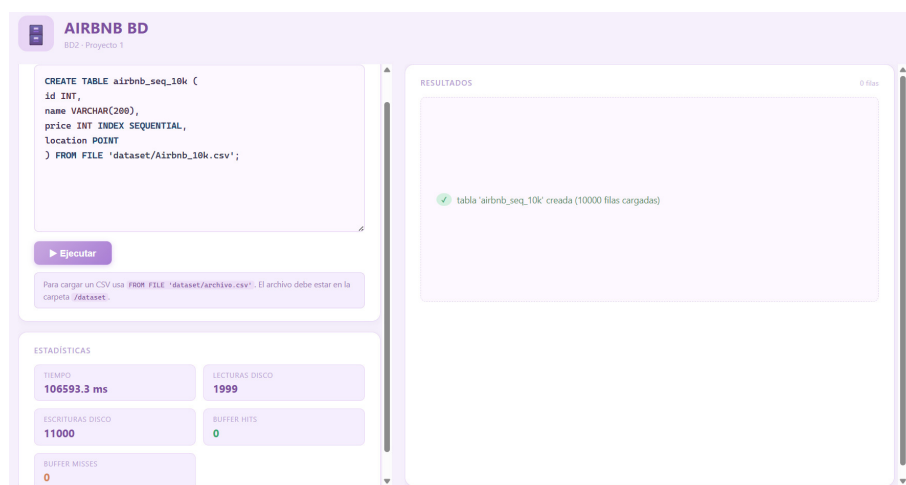


Figura 4: Inserción – Sequential File – 10k registros

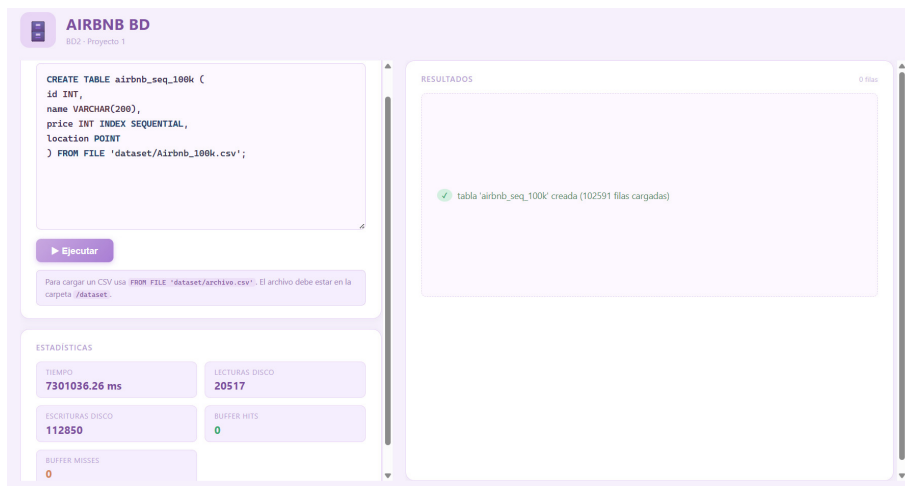


Figura 5: Inserción – Sequential File – 100k registros

B+ Tree

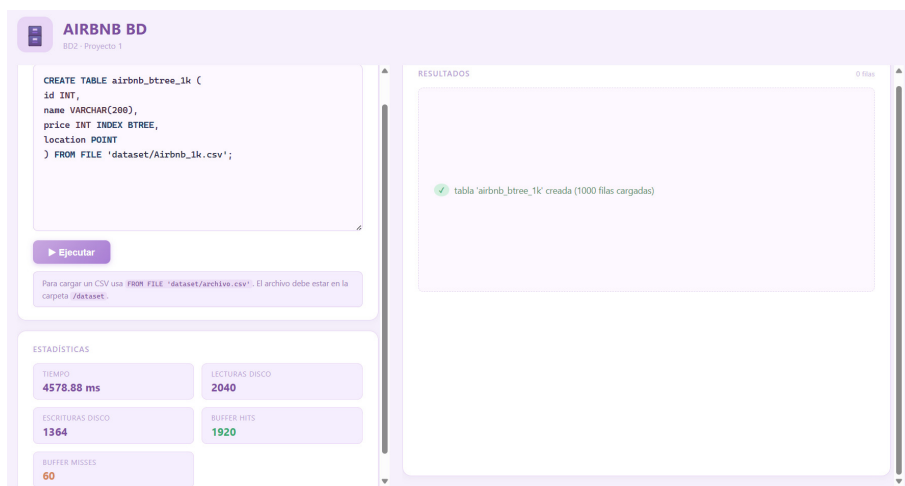


Figura 6: Inserción – B+ Tree – 1k registros

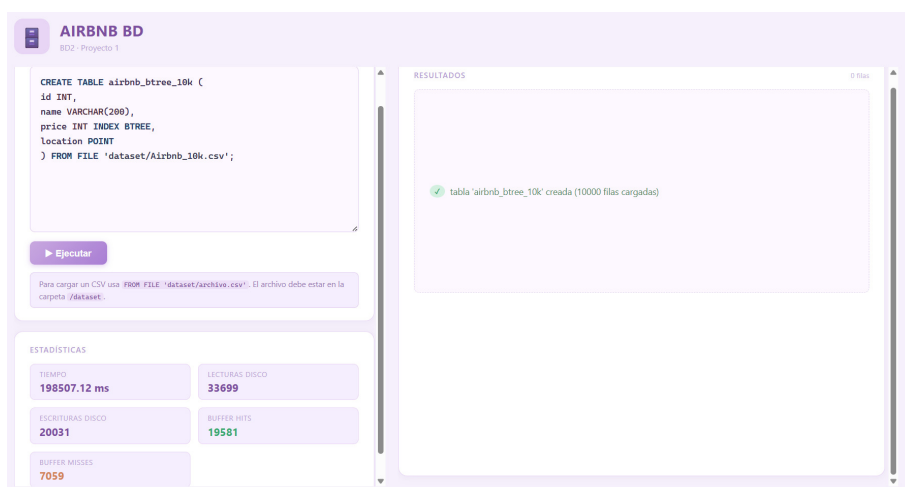


Figura 7: Inserción – B+ Tree – 10k registros

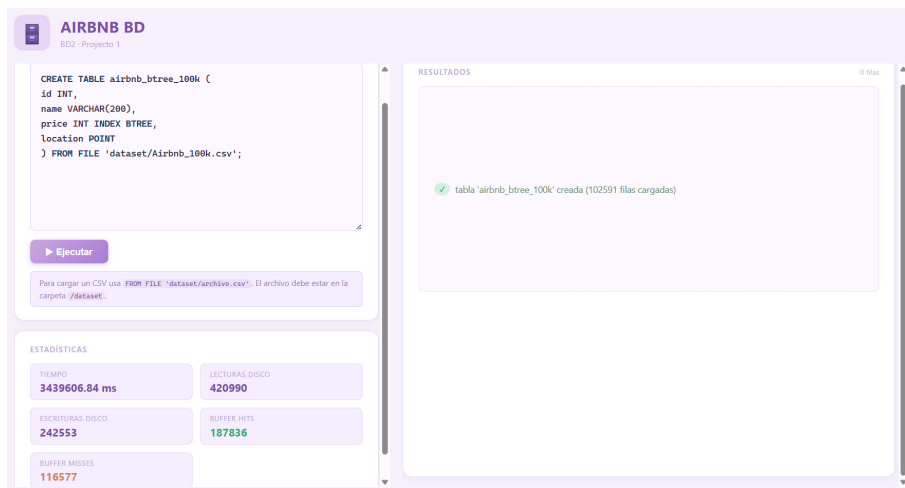


Figura 8: Inserción – B+ Tree – 100k registros

Extendible Hashing

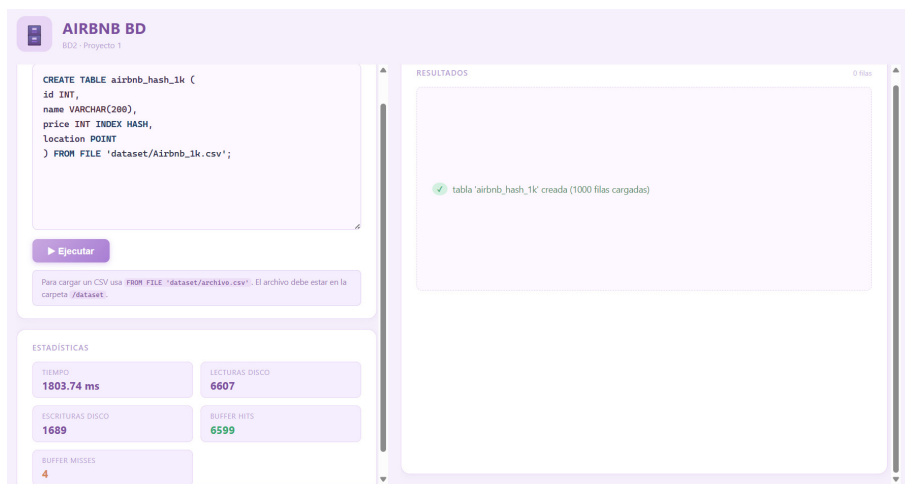


Figura 9: Inserción – Extendible Hashing – 1k registros

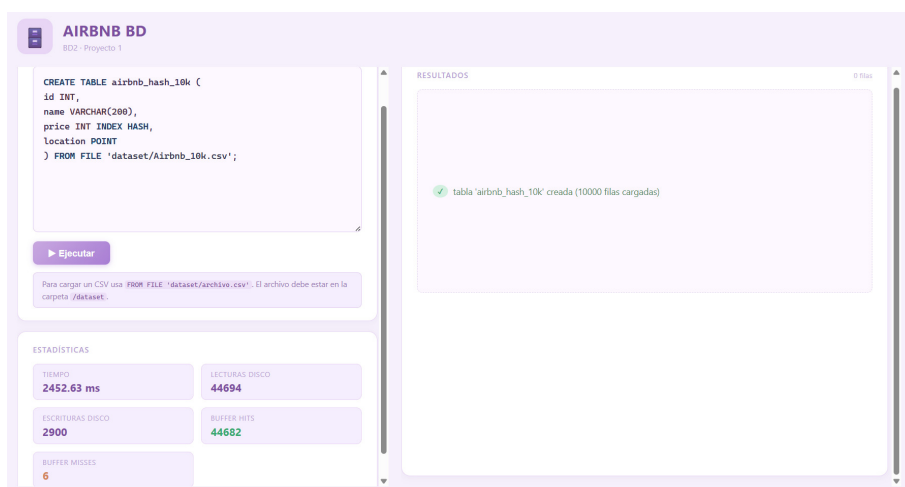


Figura 10: Inserción – Extendible Hashing – 10k registros

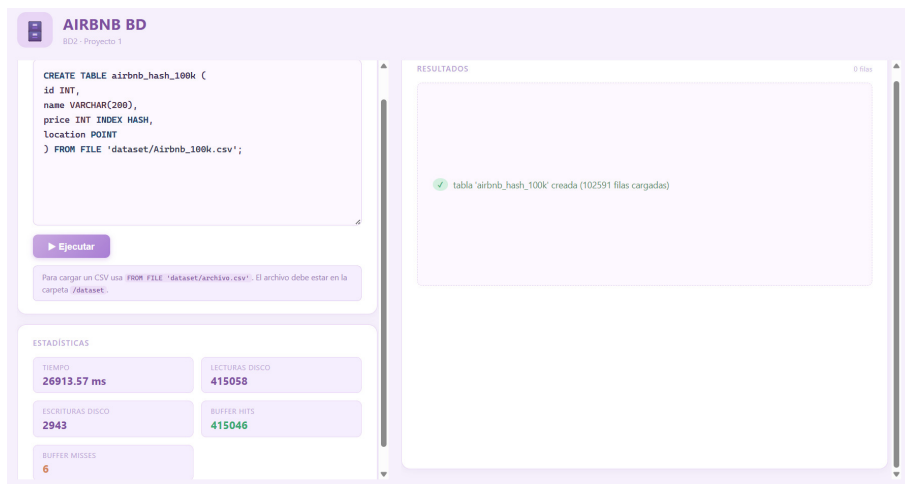


Figura 11: Inserción – Extendible Hashing – 100k registros

R-Tree

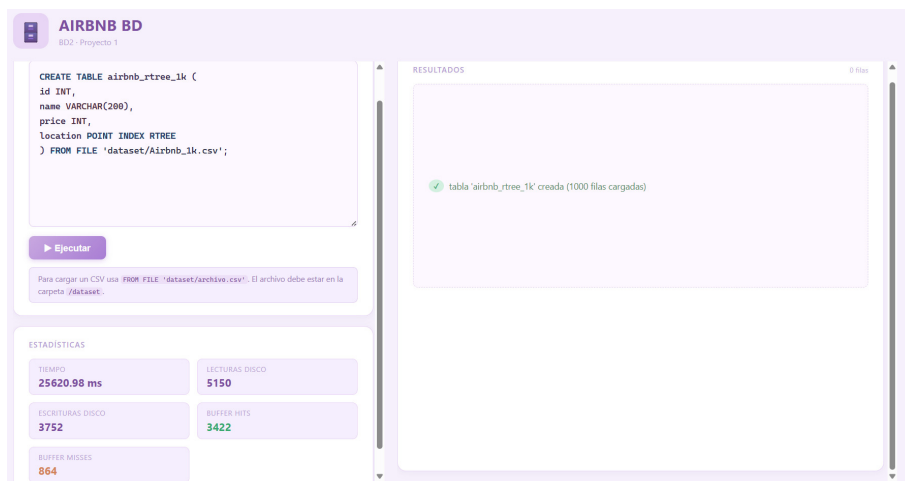


Figura 12: Inserción – R-Tree – 1k registros

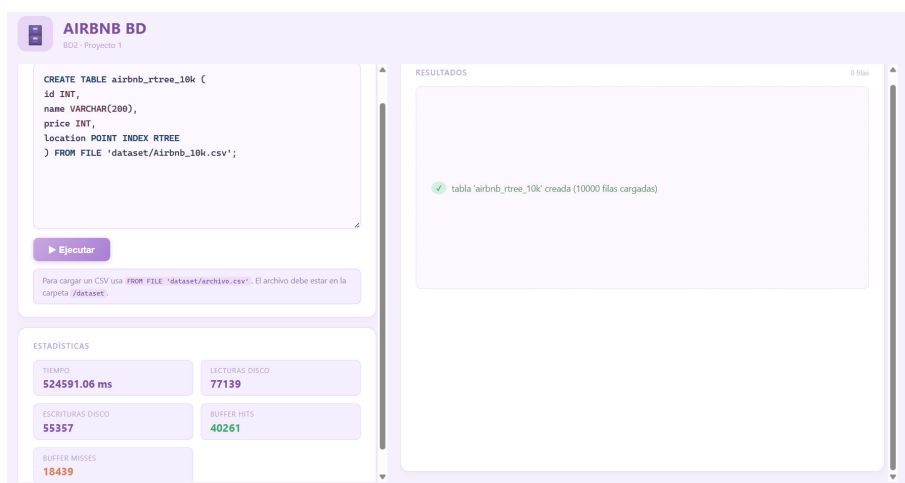


Figura 13: Inserción – R-Tree – 10k registros

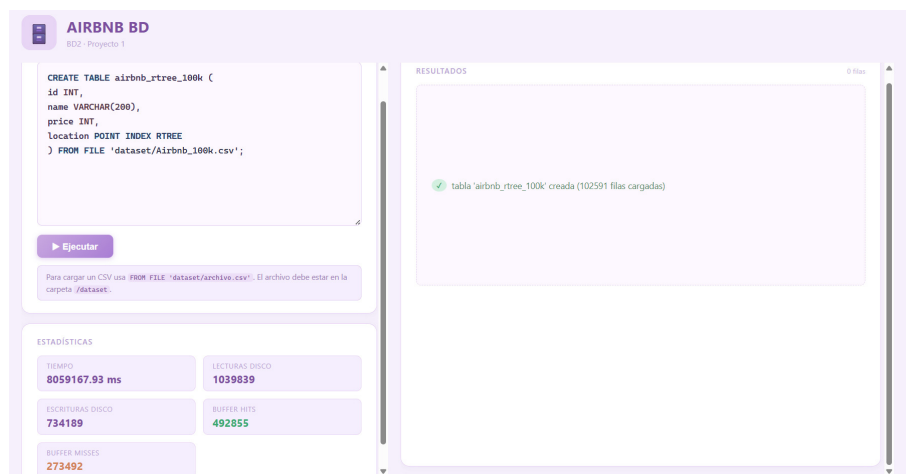


Figura 14: Inserción – R-Tree – 100k registros

Tabla de resultados

Cuadro 19: Resultados experimentales de inserción

Índice	Tamaño	Tiempo (ms)	Lecturas	Escrituras
B+Tree	1k	4578.88	2040	1364
B+Tree	10k	198507.12	33699	20031
B+Tree	100k	3439606.84	420990	242553
R-Tree	1k	25620.98	5150	3752
R-Tree	10k	524591.06	77139	55357
R-Tree	100k	8059167.93	1039839	734189
Hash	1k	1803.74	6607	1689
Hash	10k	2452.63	44694	2900
Hash	100k	26913.57	415058	2943
Sequential	1k	5147.78	199	1100
Sequential	10k	106593.30	1999	11000
Sequential	100k	7301036.26	20517	112850

Gráficos comparativos de inserción

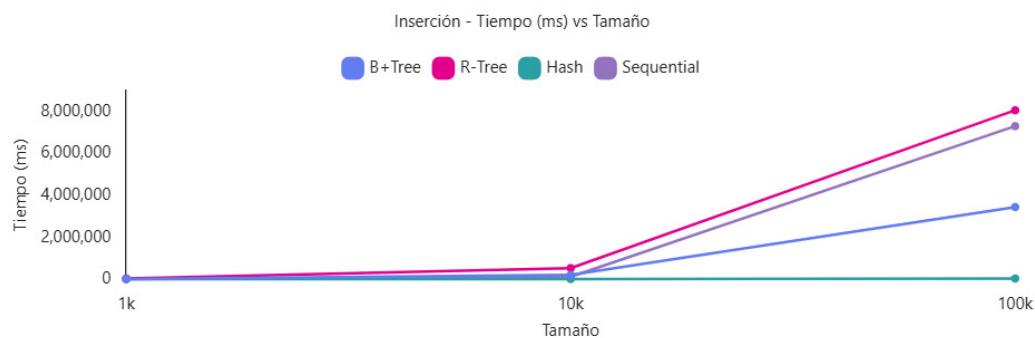


Figura 15: Inserción – Comparativa de tiempo de ejecución

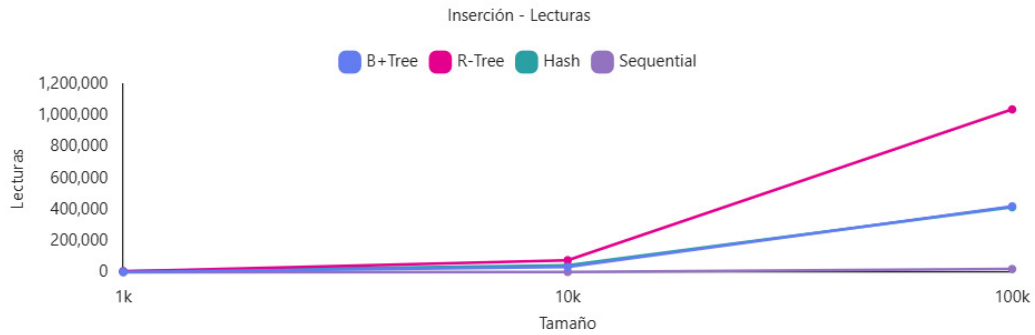


Figura 16: Inserción – Comparativa de lecturas a disco

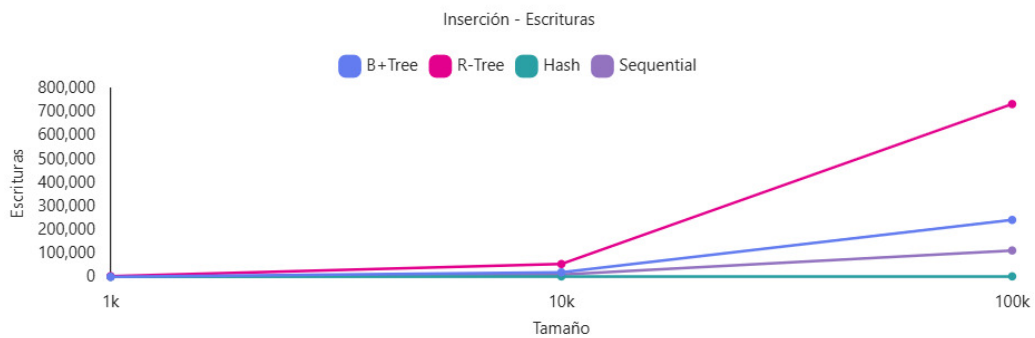


Figura 17: Inserción – Comparativa de escrituras a disco

6.2.2. Búsqueda puntual

Sequential File

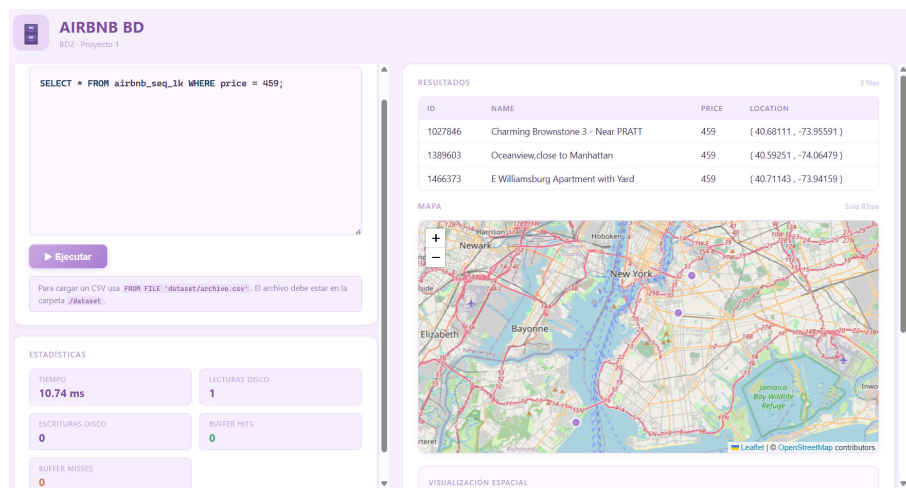


Figura 18: Búsqueda puntual – Sequential File – 1k registros

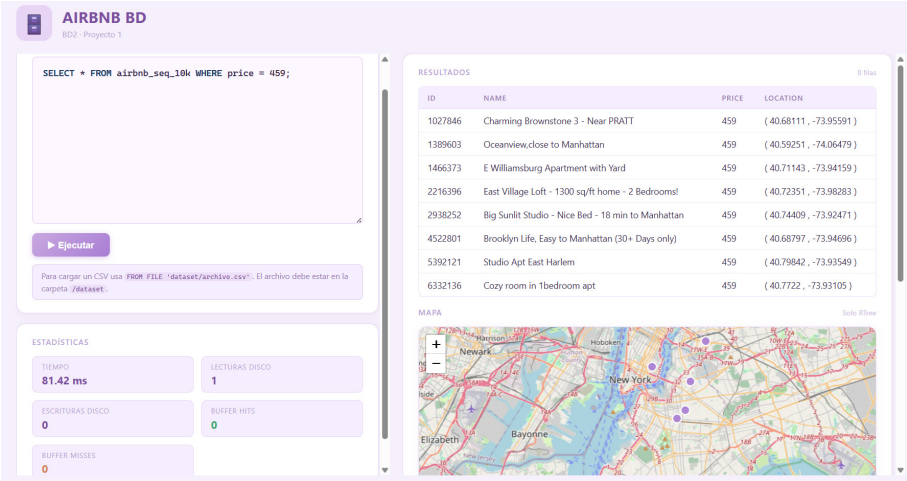


Figura 19: Búsqueda puntual – Sequential File – 10k registros

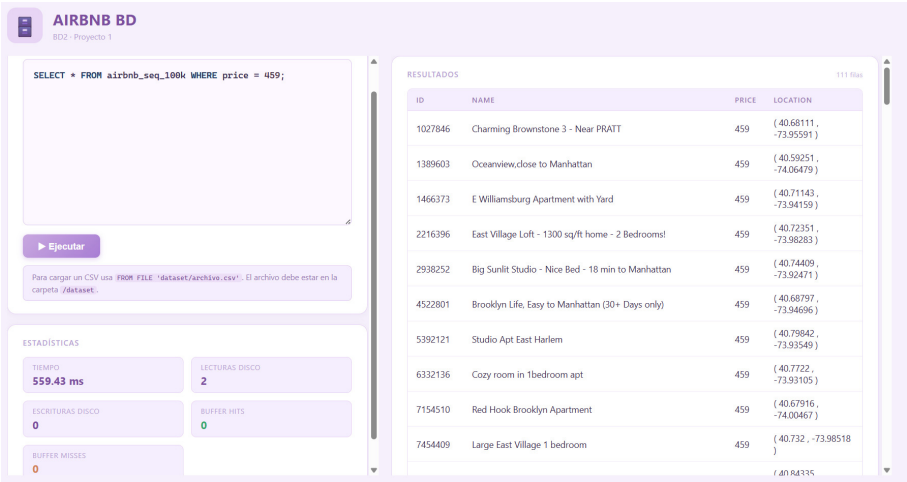


Figura 20: Búsqueda puntual – Sequential File – 100k registros

B+ Tree

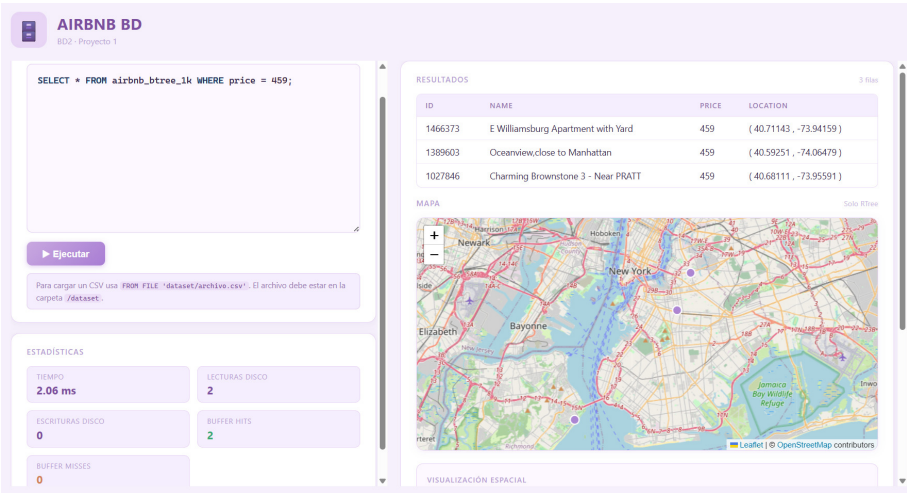


Figura 21: Búsqueda puntual – B+ Tree – 1k registros

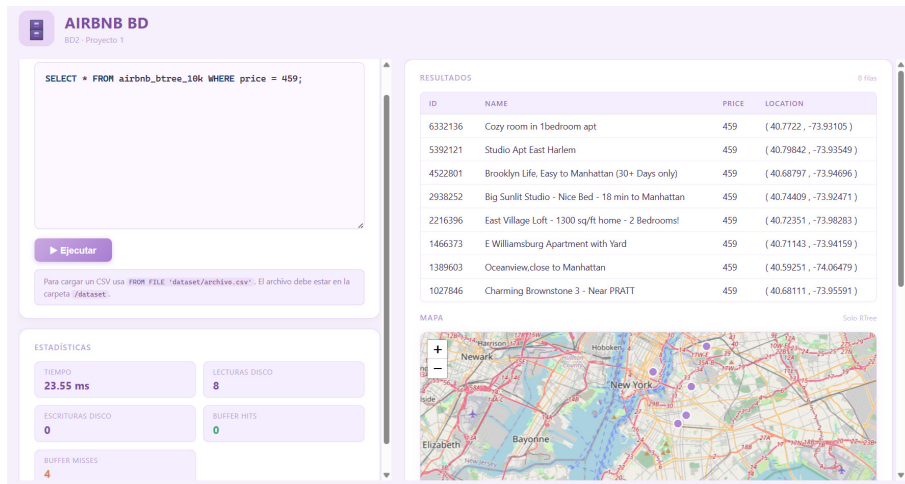


Figura 22: Búsqueda puntual – B+ Tree – 10k registros

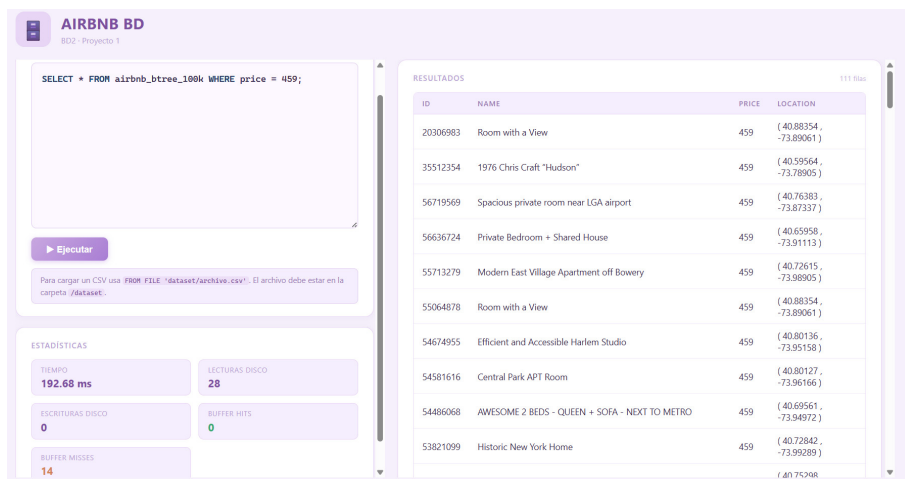


Figura 23: Búsqueda puntual – B+ Tree – 100k registros

Extendible Hashing

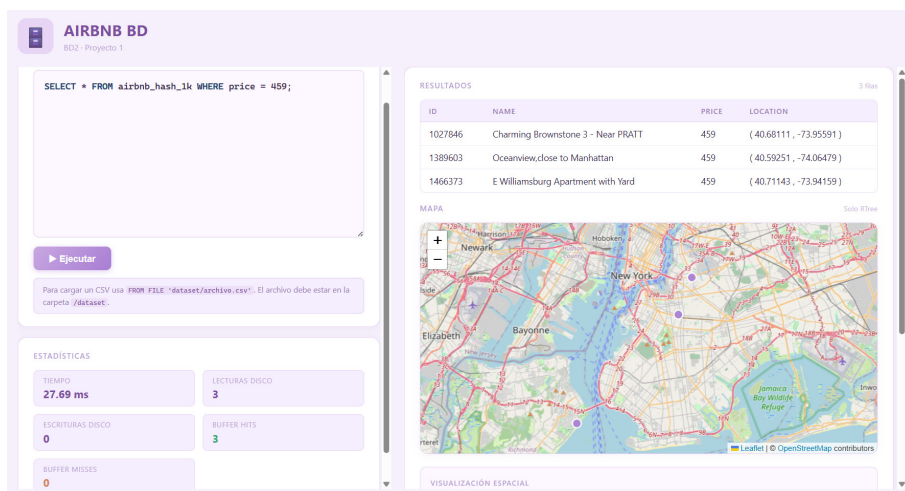


Figura 24: Búsqueda puntual – Extendible Hashing – 1k registros

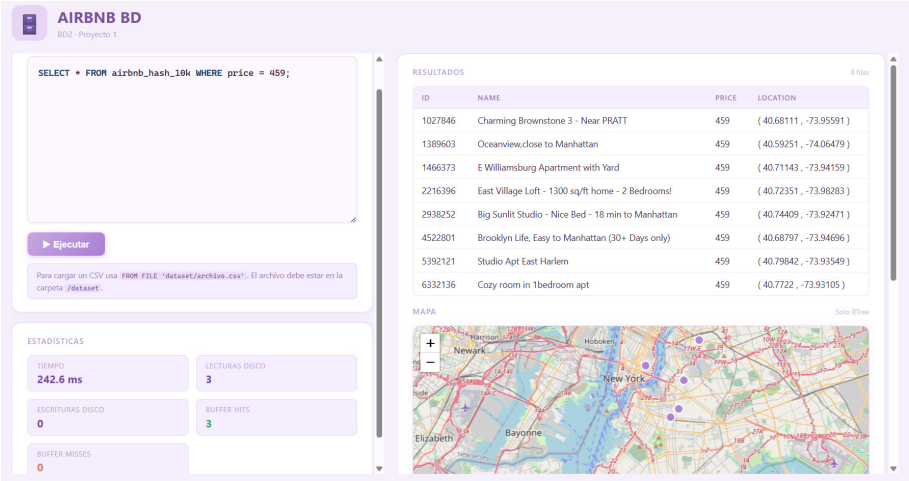


Figura 25: Búsqueda puntual – Extendible Hashing – 10k registros

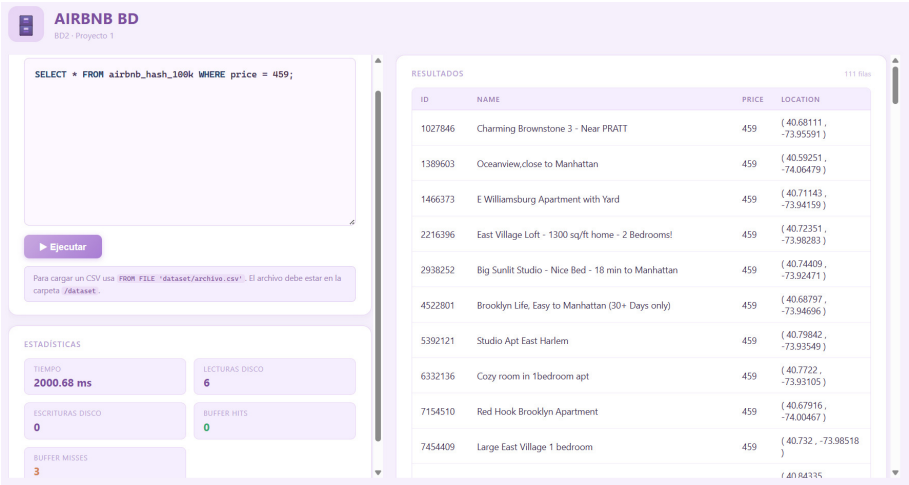


Figura 26: Búsqueda puntual – Extendible Hashing – 100k registros

Tabla de resultados

Cuadro 20: Resultados experimentales de búsqueda puntual

Índice	Tamaño	Tiempo (ms)	Lecturas
B-Tree	1k	2.06	2
B-Tree	10k	23.55	8
B-Tree	100k	192.68	28
Hash	1k	27.69	3
Hash	10k	242.60	3
Hash	100k	2000.68	6
Sequential	1k	10.74	1
Sequential	10k	81.42	1
Sequential	100k	559.43	2

Gráficos comparativos de búsqueda puntual

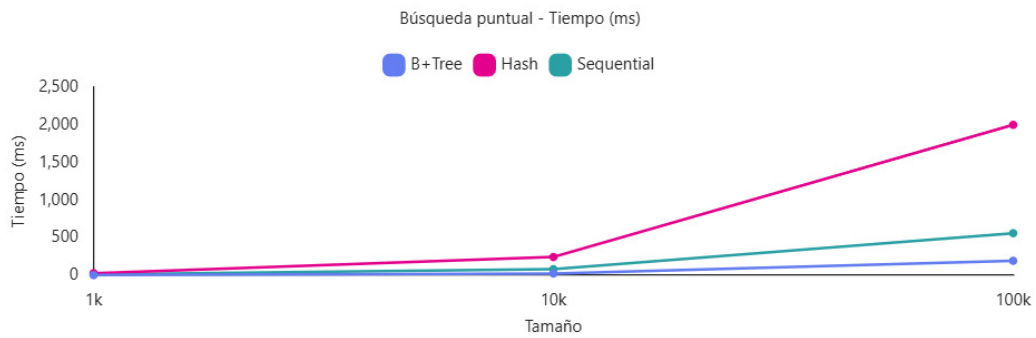


Figura 27: Búsqueda puntual – Comparativa de tiempo de ejecución

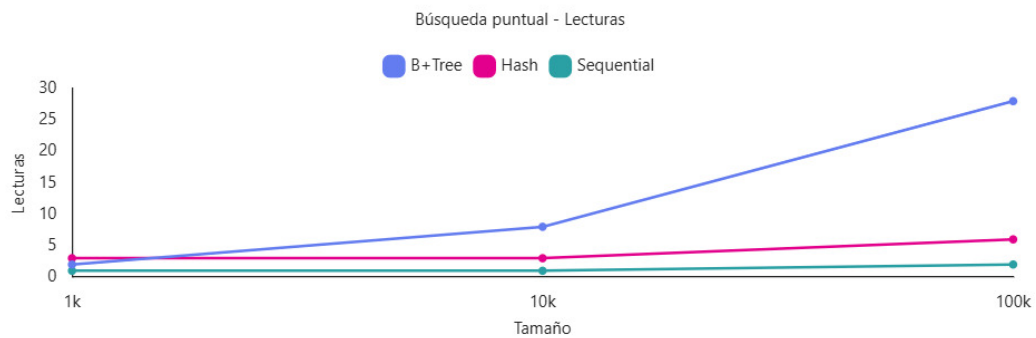


Figura 28: Búsqueda puntual – Comparativa de lecturas a disco

6.2.3. Búsqueda por rango

Sequential File

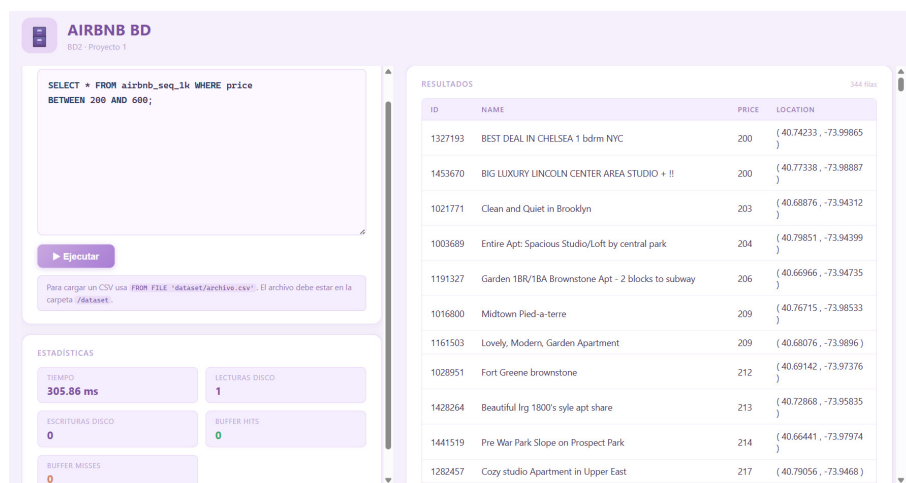


Figura 29: Búsqueda por rango – Sequential File – 1k registros

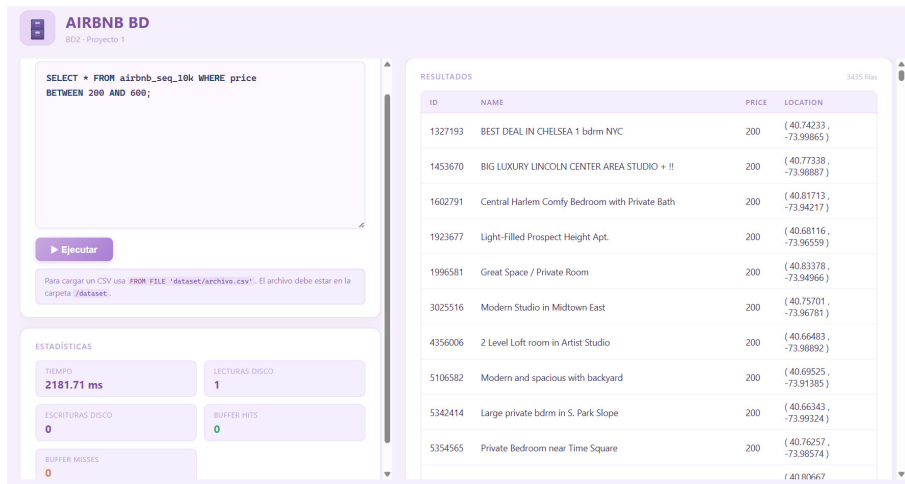


Figura 30: Búsqueda por rango – Sequential File – 10k registros

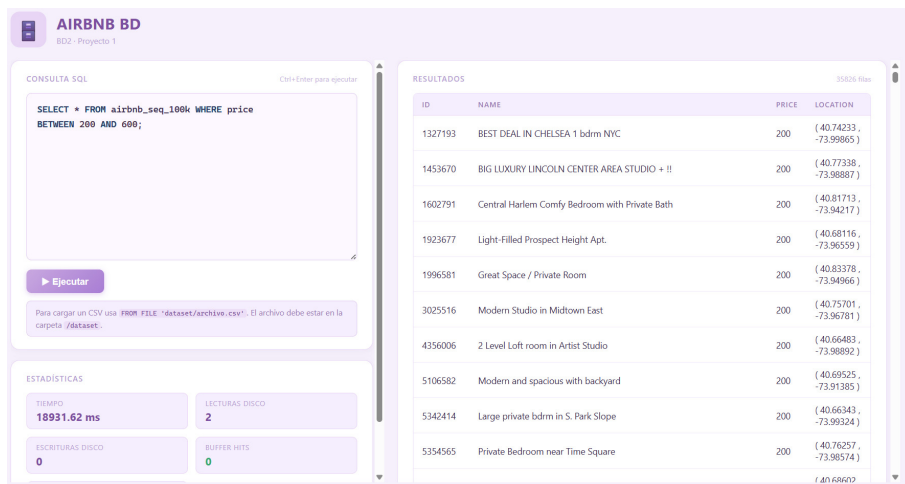


Figura 31: Búsqueda por rango – Sequential File – 100k registros

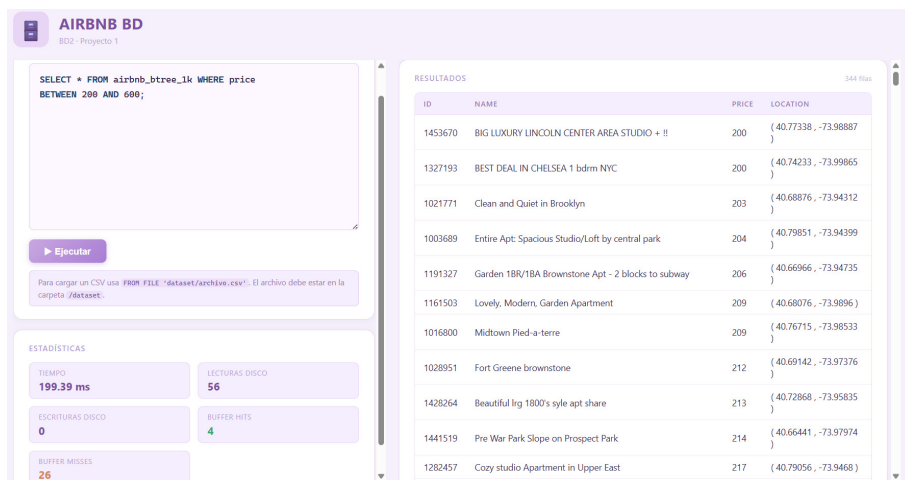


Figura 32: Búsqueda por rango – B+ Tree – 1k registros

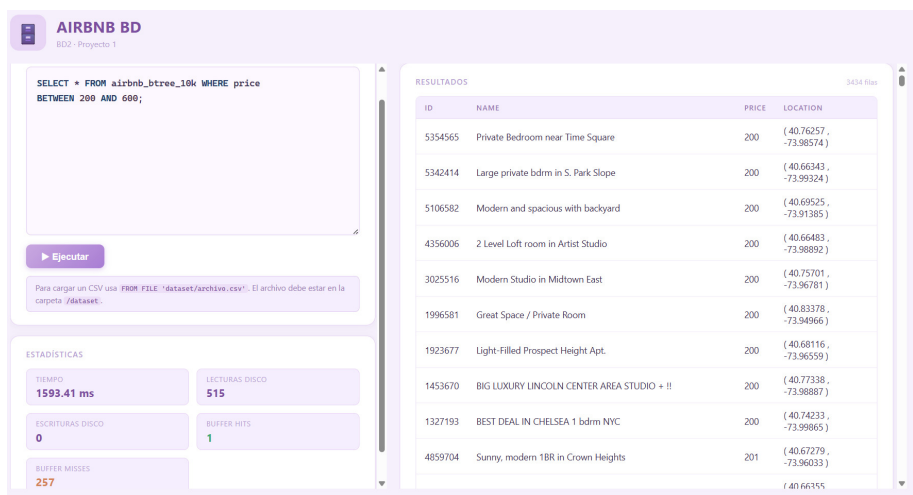


Figura 33: Búsqueda por rango – B+ Tree – 10k registros

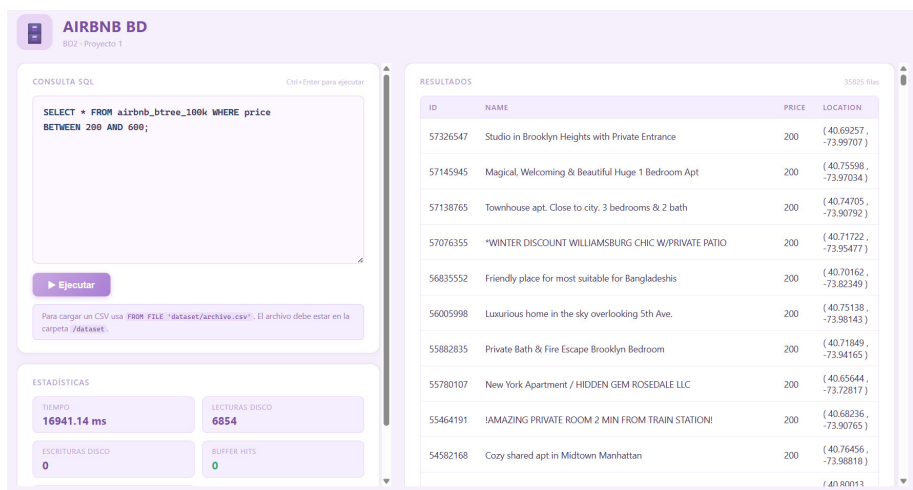


Figura 34: Búsqueda por rango – B+ Tree – 100k registros

B+ Tree Tabla de resultados

Cuadro 21: Resultados experimentales de búsqueda por rango

Índice	Tamaño	Tiempo (ms)	Lecturas
B-Tree	1k	199.39	56
B-Tree	10k	1593.41	515
B-Tree	100k	16941.14	6854
Sequential	1k	305.86	1
Sequential	10k	2181.71	1
Sequential	100k	18931.62	2

Gráficos comparativos de búsqueda por rango



Figura 35: Búsqueda por rango – Comparativa de tiempo de ejecución

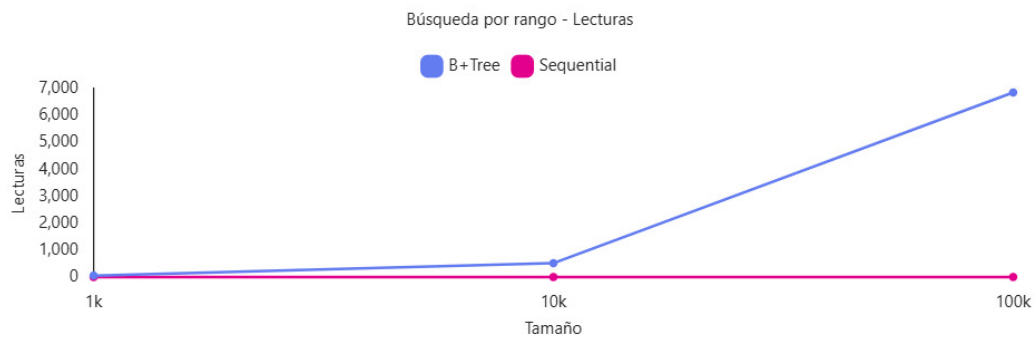


Figura 36: Búsqueda por rango – Comparativa de lecturas a disco

6.2.4. DELETE

Sequential File

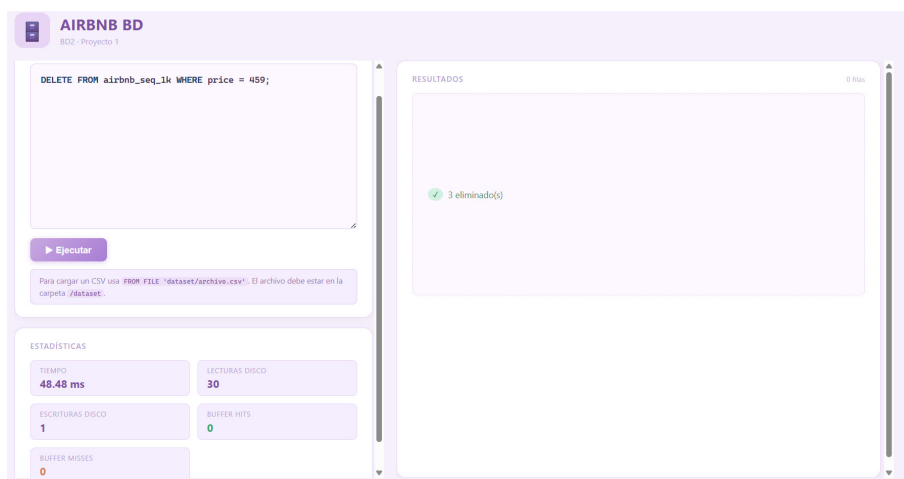


Figura 37: DELETE – Sequential File – 1k registros

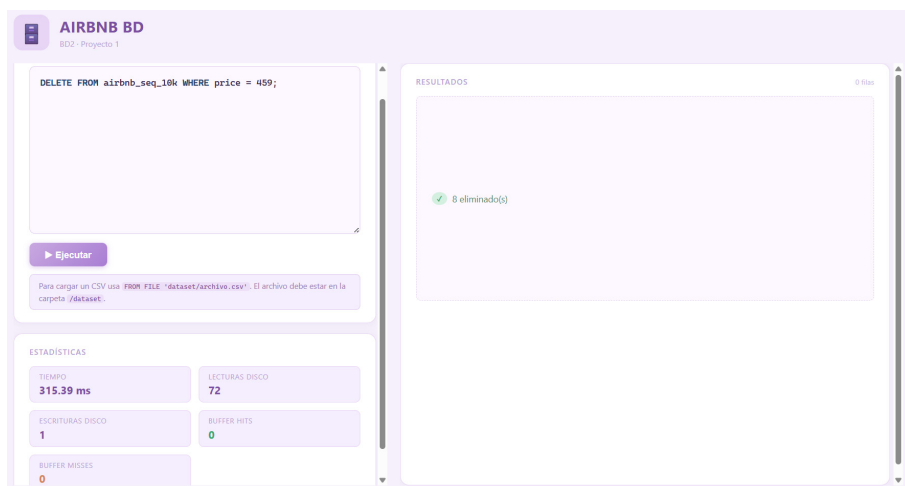


Figura 38: DELETE – Sequential File – 10k registros

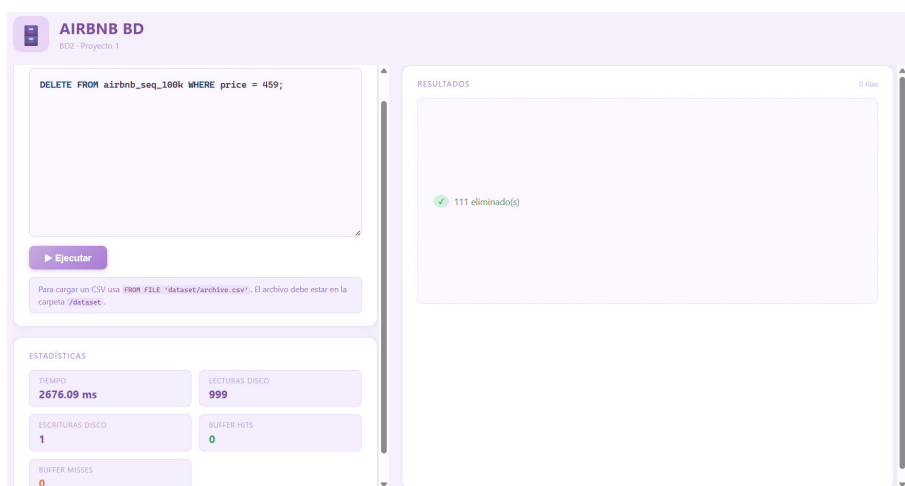


Figura 39: DELETE – Sequential File – 100k registros

B+ Tree

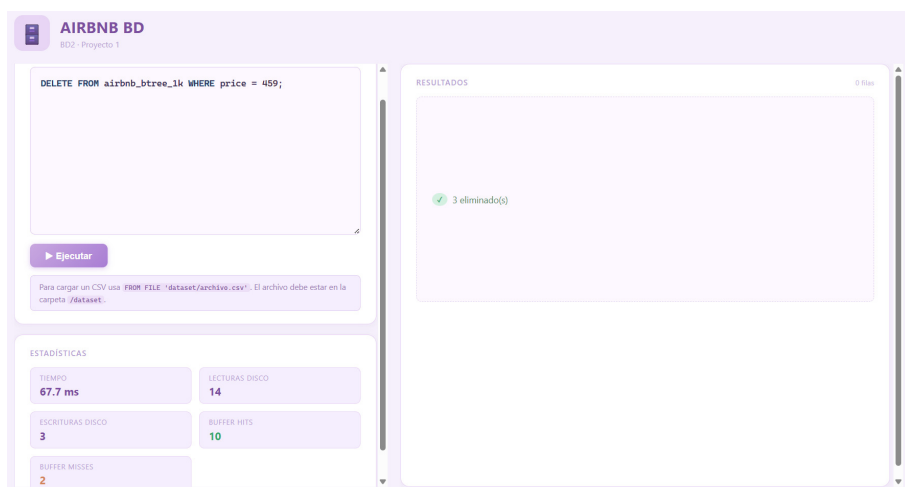


Figura 40: DELETE – B+ Tree – 1k registros

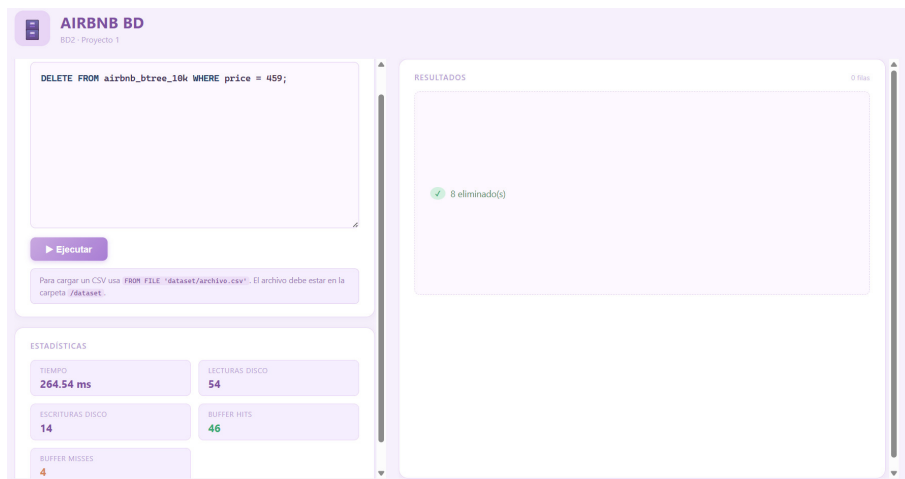


Figura 41: DELETE – B+ Tree – 10k registros

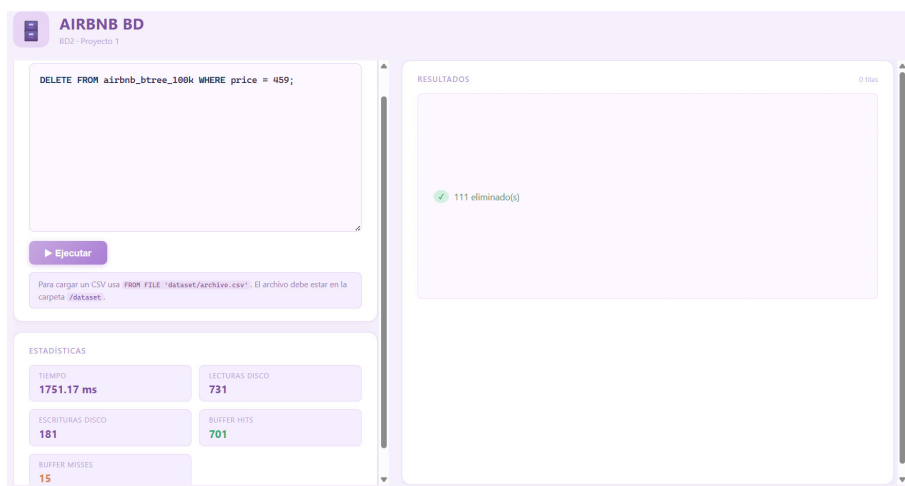


Figura 42: DELETE – B+ Tree – 100k registros

Extendible Hashing

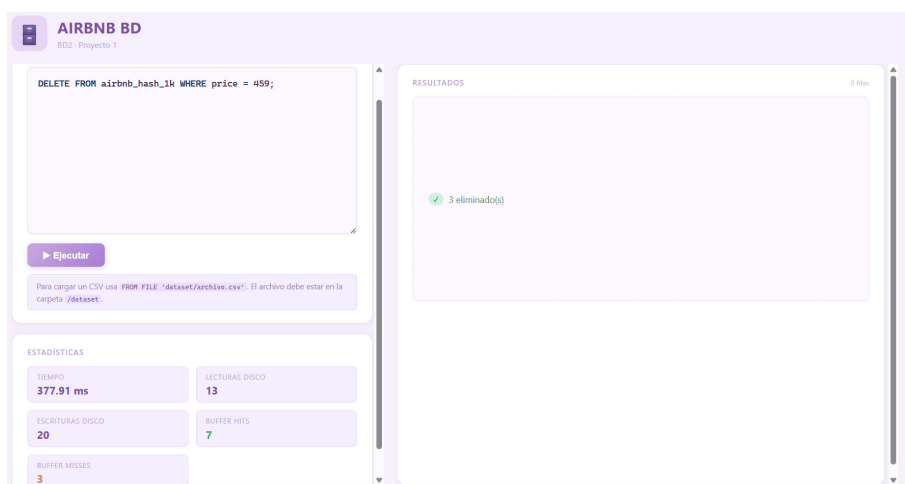


Figura 43: DELETE – Extendible Hashing – 1k registros

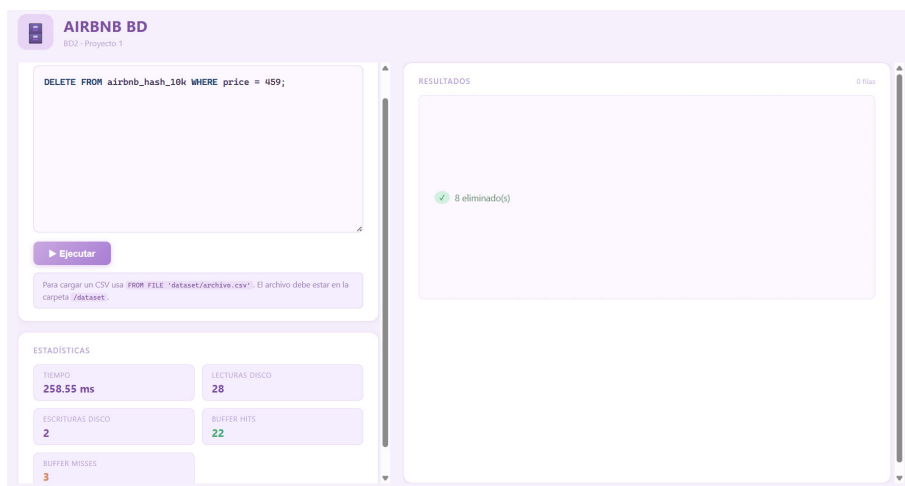


Figura 44: DELETE – Extendible Hashing – 10k registros

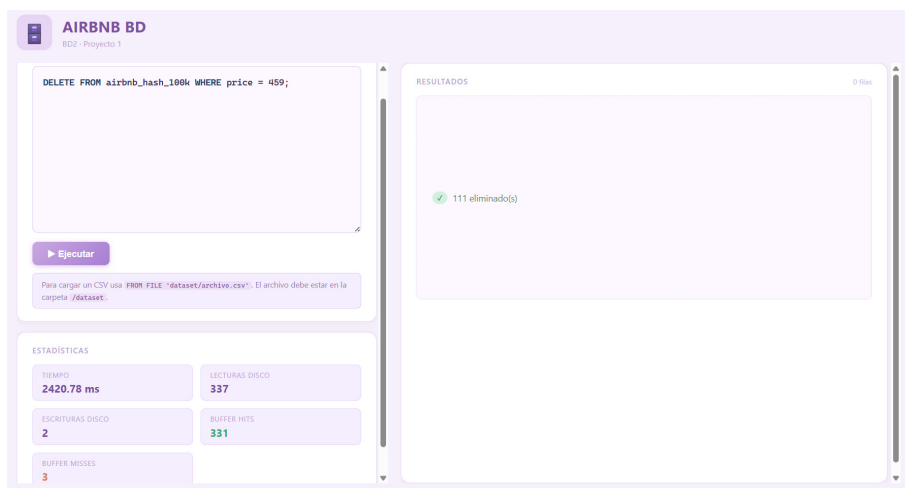


Figura 45: DELETE – Extendible Hashing – 100k registros

Tabla de resultados

Cuadro 22: Resultados experimentales de DELETE

Índice	Tamaño	Tiempo (ms)	Lecturas	Escrituras
B-Tree	1k	67.70	14	3
B-Tree	10k	264.54	54	14
B-Tree	100k	1751.17	731	181
Sequential	1k	48.48	30	1
Sequential	10k	315.39	72	1
Sequential	100k	2676.09	999	1
Hash	1k	377.91	13	20
Hash	10k	258.55	28	2
Hash	100k	2420.78	337	2

Gráficos comparativos de DELETE

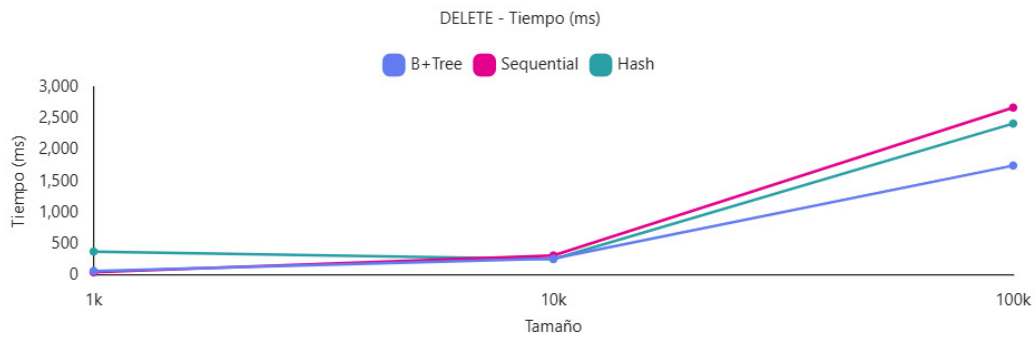


Figura 46: DELETE – Comparativa de tiempo de ejecución

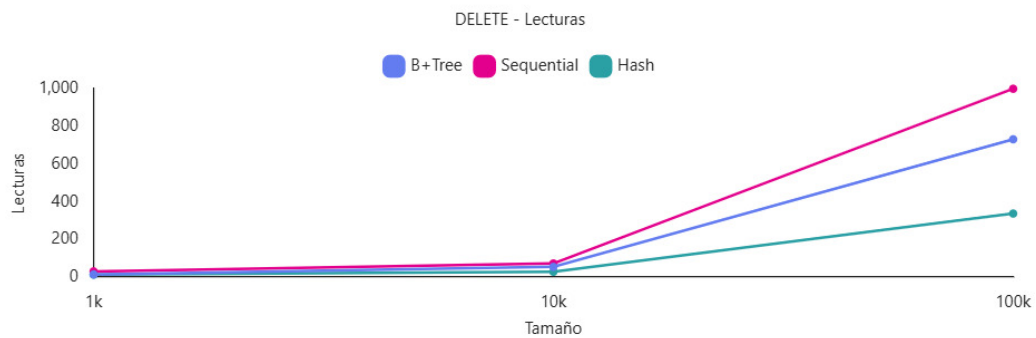


Figura 47: DELETE – Comparativa de lecturas a disco

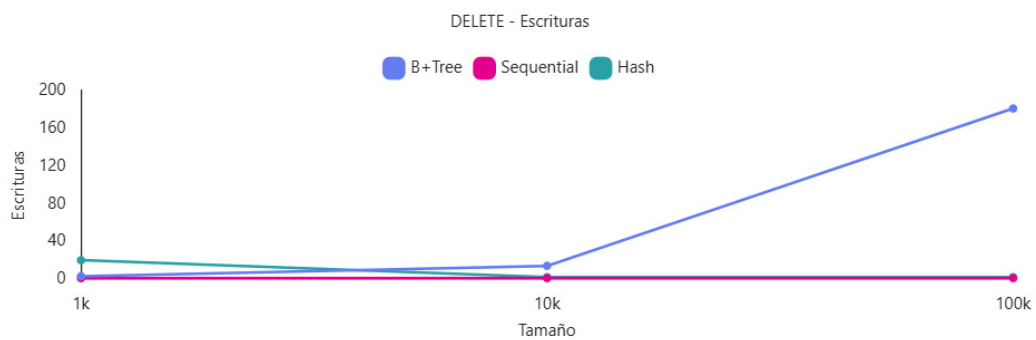


Figura 48: DELETE – Comparativa de escrituras a disco

6.2.5. R-Tree – Consultas espaciales

Búsqueda por radio

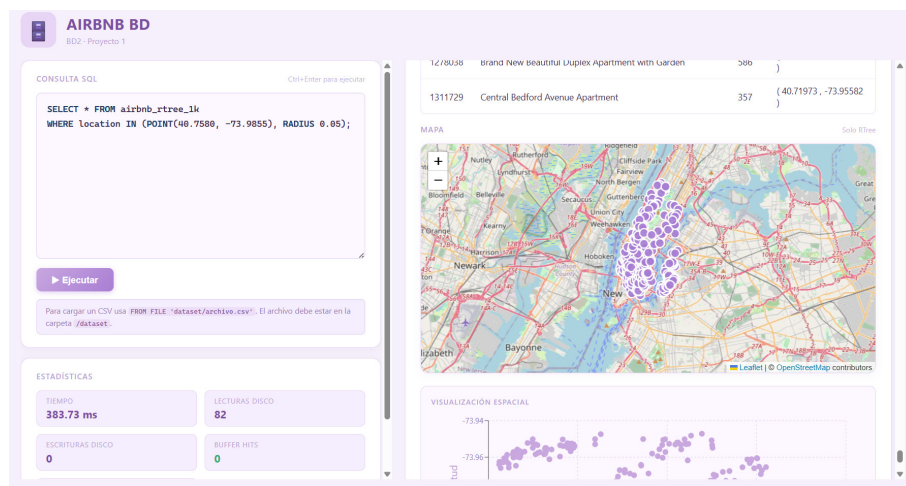


Figura 49: R-Tree – Búsqueda por radio – 1k registros

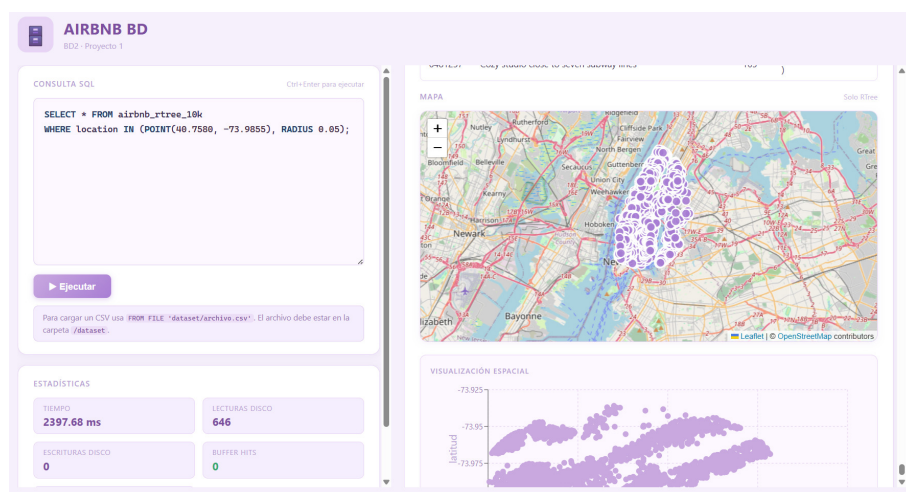


Figura 50: R-Tree – Búsqueda por radio – 10k registros

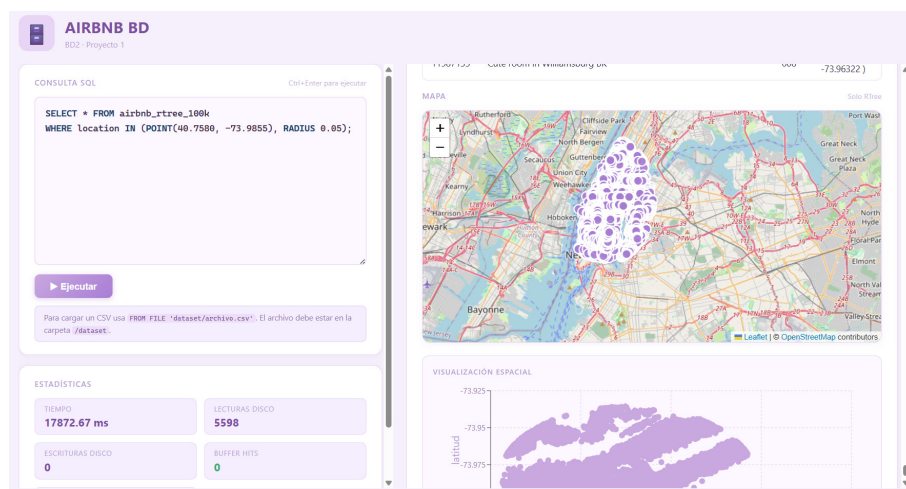


Figura 51: R-Tree – Búsqueda por radio – 100k registros

Cuadro 23: Resultados de búsqueda por radio en R-Tree

Tamaño	Tiempo (ms)	Lecturas
1k	383.73	82
10k	2397.68	646
100k	17872.67	5598

K vecinos más cercanos (KNN)

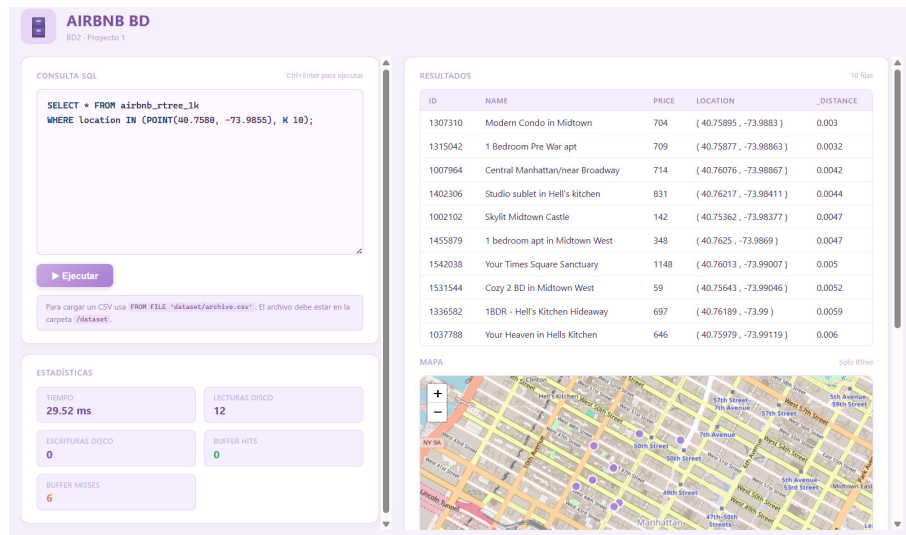


Figura 52: R-Tree – KNN – 1k registros

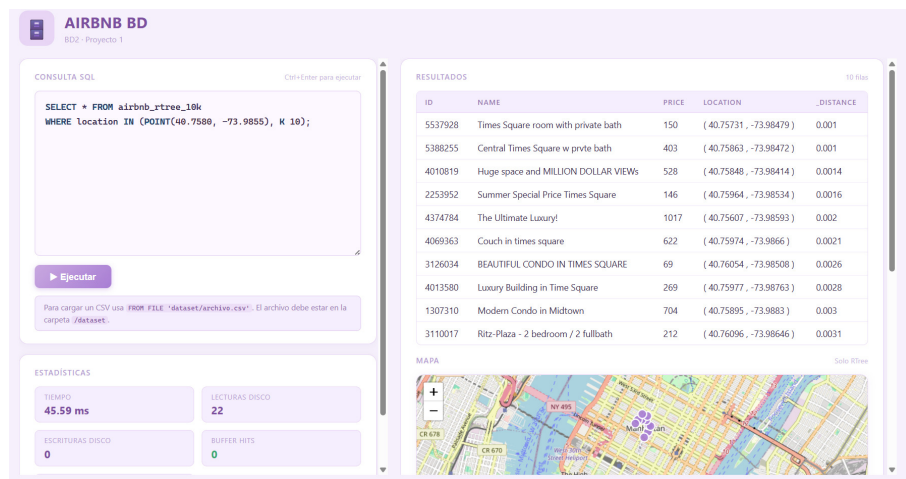


Figura 53: R-Tree – KNN – 10k registros

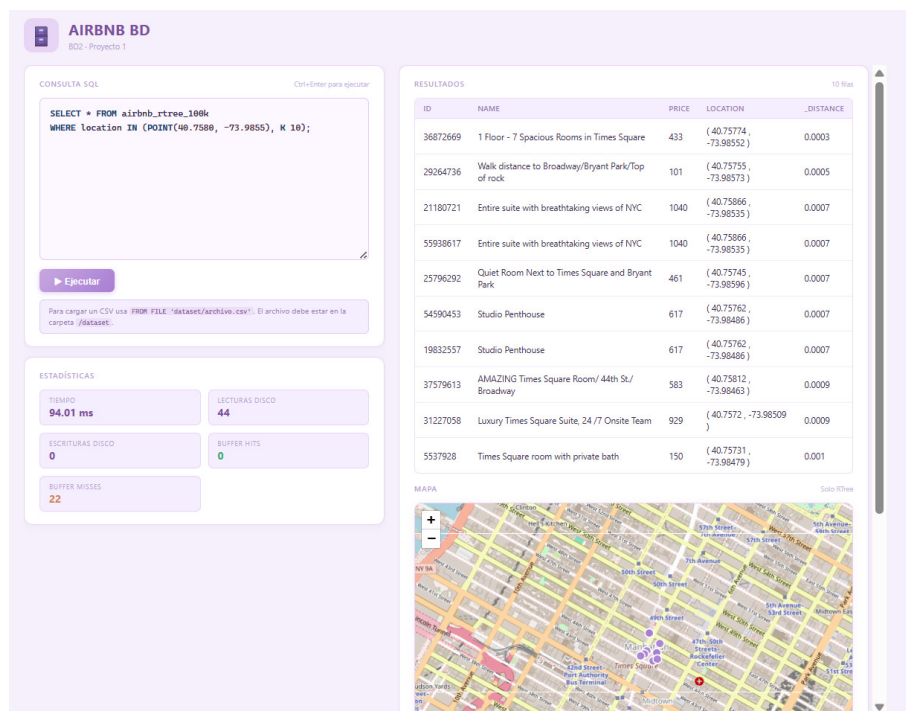


Figura 54: R-Tree – KNN – 100k registros

Cuadro 24: Resultados de KNN en R-Tree

Tamaño	Tiempo (ms)	Lecturas
1k	29.52	12
10k	45.59	22
100k	94.01	44

Gráfico comparativo de consultas espaciales

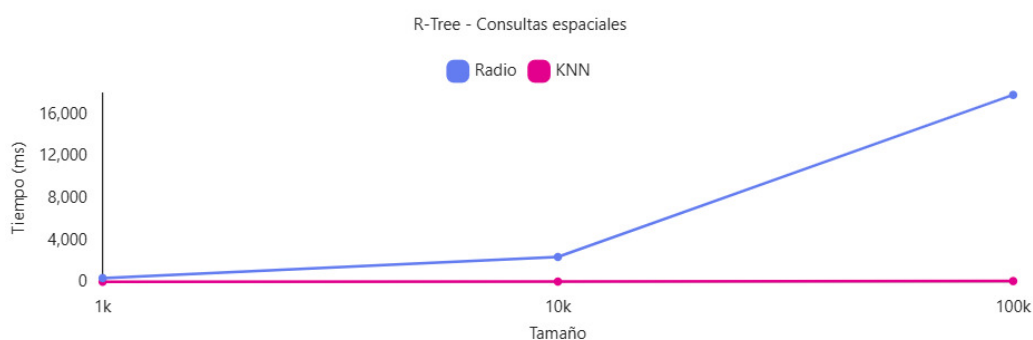


Figura 55: R-Tree – Comparativa de consultas espaciales (radio y KNN)

6.3. Discusión de resultados

Los resultados experimentales muestran diferencias importantes entre las estructuras implementadas. El Extendible Hashing presentó el mejor desempeño en búsquedas exactas debido a su acceso directo mediante función hash, manteniendo complejidad promedio

cercana a $O(1)$. Sin embargo, esta estructura no soporta eficientemente consultas por rango.

El B+ Tree mostró un mejor comportamiento en consultas BETWEEN y operaciones ordenadas, gracias a la organización secuencial de sus hojas enlazadas. Por otro lado, Sequential File presentó costos bajos de inserción inicial, aunque su rendimiento disminuye conforme crece el archivo auxiliar y se requieren procesos de rebuild.

En el caso del R-Tree, las búsquedas espaciales lograron reducir significativamente el número de páginas visitadas gracias al uso de poda mediante MBRs, especialmente en consultas KNN y búsquedas por radio.

En general, los resultados demuestran que la elección de la estructura de indexación depende directamente del tipo de consulta que se desea optimizar.

7. Pruebas de uso e interfaz gráfica

La interfaz gráfica fue desarrollada utilizando React + Vite para el frontend y FastAPI para el backend.

La aplicación incluye:

- Editor SQL con resaltado de sintaxis
- Panel de resultados tabulares
- Visualización espacial mediante Leaflet
- Estadísticas de I/O en tiempo real

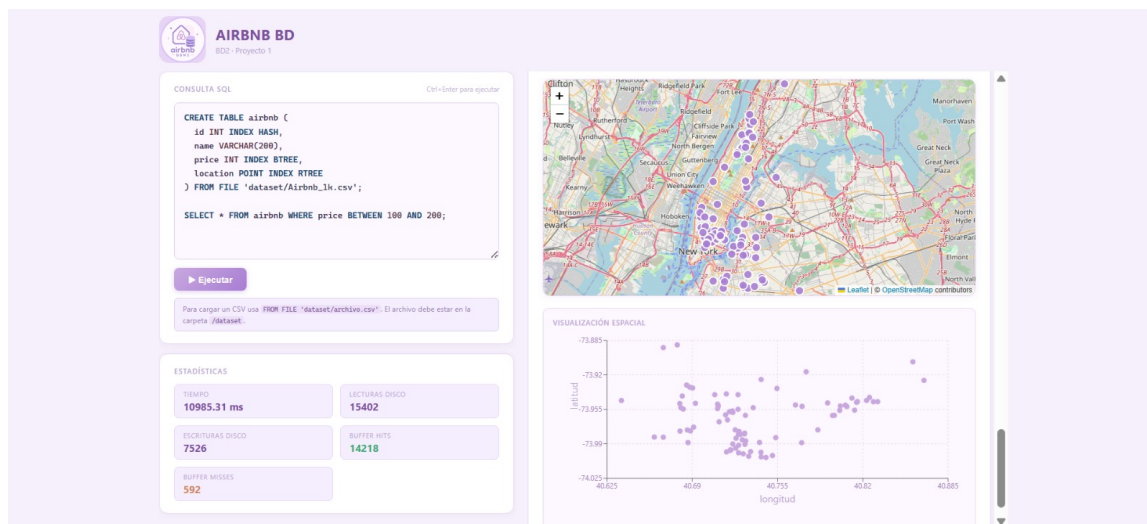


Figura 56: Interfaz gráfica del sistema

8. Algoritmos de memoria externa

8.1. External Sort

El External Sort implementado organiza archivos de registros de tamaño fijo utilizando ordenamiento externo basado en páginas de memoria. Este algoritmo permite ordenar datasets que no necesariamente caben por completo en memoria RAM.

El algoritmo se divide en dos fases principales:

1. Creación de runs ordenados.
2. Mezcla o merge de runs.

En la primera fase, el sistema lee bloques de páginas desde el archivo de entrada utilizando un buffer limitado en memoria. Los registros cargados se almacenan temporalmente en RAM, se ordenan según la clave definida y luego se escriben en archivos temporales llamados runs. Cada run contiene registros ordenados localmente.

En la segunda fase, los runs generados se combinan progresivamente mediante un proceso de k-way merge. Para ello, se utiliza un min-heap que permite seleccionar eficientemente el menor registro entre varios runs simultáneamente. El algoritmo continúa fusionando runs hasta obtener un único archivo final completamente ordenado.

La implementación trabaja sobre páginas de 4096 bytes y administra las operaciones de lectura y escritura mediante FileManager. Además, el sistema calcula estadísticas como cantidad de pasadas, runs iniciales y registros procesados.

8.2. External Hashing para Join

El External Hashing Join es un algoritmo de join orientado a consultas por igualdad, también llamadas equi-joins. Este utiliza almacenamiento externo en disco para construir una estructura hash temporal. A diferencia del Hash Join tradicional, donde toda la tabla hash se mantiene en memoria RAM, esta implementación crea buckets en páginas almacenadas físicamente en disco. Esto permite trabajar con conjuntos de datos más grandes y mantener consistencia con el modelo paginado implementado en el resto del motor.

Modelo de almacenamiento externo

La implementación utiliza un único archivo binario compartido:

```
data/external_hashing/join_*/external_hashing.dat
```

El directorio `join_*` es generado temporalmente durante la ejecución del join. A diferencia de implementaciones simples donde cada bucket ocupa un archivo independiente, en este caso todos los buckets comparten el mismo archivo físico. El sistema mantiene en memoria un directorio que indica qué páginas contienen entradas pertenecientes a cada bucket.

Formato de página

Cada página tiene un header y luego una secuencia de entradas.

Cuadro 25: Formato de página en External Hashing Join

Campo	Descripción
used_bytes	Header de 2 bytes que indica la cantidad de bytes ocupados dentro de la página.
bucket_id	Identificador del bucket lógico al que pertenece la entrada.
payload_size	Tamaño del payload almacenado.
payload	Par (key , row) serializado en binario.

El payload representa la información útil almacenada dentro de cada entrada. En esta implementación, corresponde al par (**key**, **row**), donde **key** es la clave de join y **row** es la fila completa asociada.

Serialización mediante pack/unpack

Cada payload almacenado comienza con un tag binario que identifica su tipo. Los tags usados son:

Cuadro 26: Tags utilizados en External Hashing Join

Tag	Tipo
N	NULL
B	BOOL
I	INT
F	FLOAT
S	STRING
D	DICTIONARY

Esto permite reconstruir correctamente los datos durante la lectura. En el caso de strings, se almacena el tag, el tamaño del string y los bytes. En el caso de diccionarios, se almacenan las filas completas como diccionarios.

Función hash

Para calcular el bucket se utiliza SHA-256 sobre la clave serializada:

```
digest = hashlib.sha256(raw_key).digest()
bucket_id = int.from_bytes(digest[:8], "big") % bucket_count
```

Se eligió SHA-256 porque genera una distribución uniforme, es estable entre ejecuciones y evita la randomización del hash interno de Python.

Fase 1: Build

En la fase build se elige la tabla más pequeña, es decir, aquella con menor cantidad de registros. Esto reduce la cantidad de páginas temporales necesarias y minimiza el costo de búsqueda posterior.

Para cada fila se realiza el siguiente proceso:

1. Se calcula **bucket_id** utilizando la función hash.
2. Se serializa (**key**, **row**) usando pack.

3. Se construye la entrada binaria.
4. Se busca una página con espacio disponible.
5. Se escribe la entrada en la página.
6. Se actualiza el directorio en memoria.

Fase 2: Probe

En la fase probe se recorre la segunda tabla. Para cada fila:

1. Se calcula nuevamente el `bucket_id`.
2. Se consultan las páginas asociadas a ese bucket.
3. Se leen solo dichas páginas.
4. Se filtran entradas de otros buckets.
5. Se reconstruye (`key`, `row`) del bucket.
6. Se comparan las claves reales.
7. Si coincide, se genera el resultado del join.

Manejo de colisiones

Si dos claves distintas caen en el mismo bucket, ambas se almacenan dentro de las páginas asociadas a dicho bucket. Durante la búsqueda, primero se filtra por `bucket_id` y luego se compara la clave real almacenada, es decir, `stored_key == key`. Esto permite manejar colisiones sin perder registros ni retornar resultados incorrectos.

Integración con FileManager y BufferManager

El External Hashing reutiliza la misma infraestructura de almacenamiento utilizada por el resto del motor. Las páginas del archivo `external_hashing.dat` son leídas y escritas mediante FileManager. Cuando existe un BufferManager, las páginas pasan primero por el buffer pool antes de acceder al disco, permitiendo reutilizar páginas recientemente usadas y reducir lecturas innecesarias. Gracias a esta integración, el algoritmo también participa en las estadísticas generales del sistema, como buffer hits, buffer misses, disk reads y disk writes.

Cuadro 27: Complejidad de External Hashing Join

Operación	Complejidad promedio
Build	$O(n_{\text{build}})$
Probe	$O(n_{\text{probe}})$
Join completo	$O(n_{\text{build}} + n_{\text{probe}} + \text{output})$
Espacio temporal	$O(n_{\text{build}})$

8.3. Group By y Count(*)

El motor de la DBMS sigue el pipeline clásico de tres capas:

SQL → Scanner → Parser → Engine → Resultado

Cada capa opera de manera independiente. Esta separación permitió implementar **GROUP BY** modificando únicamente tres archivos, sin alterar los índices ni el DBManager.

8.3.1. Gramática soportada

La gramática presentada soporta consultas **SELECT** básicas sobre una tabla, permitiendo seleccionar todas las columnas mediante ***** o una lista de columnas y expresiones separadas por comas. Además, incluye soporte para la función de agregación **COUNT(*)**, lo que permite realizar conteos de registros. También admite la cláusula **GROUP BY**, utilizada para agrupar los resultados según una columna específica, facilitando consultas agregadas como conteos por categoría o por atributo.

```
select_stmt ::= SELECT select_list FROM IDENTIFIER
              [ JOIN IDENTIFIER ON column_ref = column_ref ]
              [ WHERE condition ]
              [ ORDER BY IDENTIFIER [ ASC | DESC ] ]
              [ GROUP BY IDENTIFIER ]
              SEMICOLON

select_list ::= STAR
             | select_expr { COMMA select_expr }

select_expr ::= COUNT LPAREN STAR RPAREN      (* COUNT(*) *)
              | IDENTIFIER                    (* columna simple *)
```

Figura 57: Interfaz gráfica del sistema

Dentro de las restricciones, solo se admite:

- Solo se admite **COUNT(*)** como función de agregación. **SUM**, **AVG**, **MAX**, **MIN** no están implementados.
- **GROUP BY** acepta una única columna (no listas de columnas).
- **HAVING** no está implementado.
- **GROUP BY** no puede combinarse con **JOIN** (la arquitectura lo permite, pero no se ha validado en el parser).


```

-- Full scan con GROUP BY
SELECT neighbourhood, COUNT(*) FROM airbnb GROUP BY
neighbourhood;

-- Con SELECT *
SELECT * FROM ventas GROUP BY categoria;

-- Con filtrado previo por índice BTREE
SELECT tipo, COUNT(*) FROM productos WHERE precio
BETWEEN 10 AND 60 GROUP BY tipo;

-- Con filtrado por igualdad e índice HASH
SELECT estado, COUNT(*) FROM pedidos WHERE id = 3
GROUP BY estado;

```

Figura 58: Interfaz gráfica del sistema

8.3.2. Cambios por capa

Capa léxica — scanner.py

Enum TokenType — se añadieron dos nuevos miembros al final del bloque:

```

GROUP = auto()
COUNT = auto()

```

Diccionario KEYWORDS — se registraron las formas en minúscula para que el scanner las reconozca independientemente de mayúsculas/minúsculas (el scanner normaliza a minúscula antes de buscar en KEYWORDS):

```

"group": TokenType.GROUP,
"count": TokenType.COUNT,

```

No se introdujeron ambigüedades: GROUP solo puede aparecer después de una cláusula WHERE u ORDER BY en un SELECT; COUNT solo puede aparecer en la lista de proyección de un SELECT.

Capa sintáctica — parser.py

Se añadieron dos métodos nuevos y se modificó uno:

Método `_parse_select_list`: devuelve None para SELECT *, preservando compatibilidad total con el código existente del engine; y para listas explícitas, devuelve una lista de nodos de expresión.

```

def _parse_select_list(self):
    if self._check(TokenType.STAR):
        self._advance()

```

```

        return None          # SELECT * -> sin lista explícita
    exprs = [self._parse_select_expr()]
    while self._match(TokenType.COMMA):
        exprs.append(self._parse_select_expr())
    return exprs

```

Método `_parse_select_expr`: reconoce `COUNT(*)` y lo representa como `{"kind": "COUNT_STAR"}`; cualquier otro token se interpreta como nombre de columna: `{"kind": "COLUMN", "name": nombre}`.

```

def _parse_select_expr(self) -> dict:
    if self._check(TokenType.COUNT):
        self._advance()
        self._expect(TokenType.LPAREN)
        self._expect(TokenType.STAR)
        self._expect(TokenType.RPAREN)
        return {"kind": "COUNT_STAR"}
    name = self._expect(TokenType.IDENTIFIER).value
    return {"kind": "COLUMN", "name": name}

```

Modificación de `_parse_select`: se reemplazó `self._expect(TokenType.STAR)` por `self._parse_select_expr()` al inicio, y se añadió el bloque de `GROUP BY` al final, antes de construir el nodo:

```

# group by a adido - parsear lista de columnas o *
columns = self._parse_select_list()

```

```

# ... (JOIN, WHERE, ORDER BY sin cambios) ...

```

```

# group by a adido - parsear GROUP BY al final
group_by = None
if self._match(TokenType.GROUP):
    self._expect(TokenType.BY)
    group_by = self._expect(TokenType.IDENTIFIER).value

```

Se incluyeron los nuevos campos en el nodo AST resultante. Por ejemplo, para la consulta:

```
SELECT neighbourhood, COUNT(*) FROM airbnb GROUP BY neighbourhood;
```

el AST generado es:

```

{
  "type": "SELECT",
  "table": "airbnb",
  "columns": [
    {"kind": "COLUMN", "name": "neighbourhood"},
    {"kind": "COUNT_STAR"}
  ],
  "join": null,
  "condition": null,
  "order_by": null,
  "group_by": "neighbourhood"
}

```

Capa de ejecución — engine.py

Se añadió soporte para `GROUP BY` dentro del método `_select`, invocando `_apply_group_by` después de aplicar `ORDER BY`. El proceso sigue este orden: recuperar filas, ordenar si existe `ORDER BY` y finalmente agrupar los registros. El método `_apply_group_by` recorre las filas, agrupa los valores según la columna indicada y calcula la cantidad de registros por grupo mediante `COUNT(*)`, devolviendo una lista con los resultados agregados.

8.3.3. Flujo de ejecución end-to-end

El flujo de ejecución inicia cuando el Scanner realiza el análisis léxico de la consulta SQL y la transforma en una secuencia de tokens reconocibles por el sistema. Posteriormente, el Parser analiza dichos tokens siguiendo la gramática definida y construye un Árbol de Sintaxis Abstracta (AST), identificando las columnas seleccionadas, la función `COUNT(*)` y la cláusula `GROUP BY categoria`. Finalmente, el Engine ejecuta la consulta obteniendo todos los registros de la tabla `ventas`, agrupándolos según el valor de la columna `categoria` y calculando la cantidad de registros por grupo. Como resultado, se genera una colección de filas agregadas con el conteo correspondiente para cada categoría.

SQL: "SELECT categoria , COUNT(*) FROM ventas GROUP BY categoria;"

1. `Scanner.tokenize()`
→ `[SELECT, IDENTIFIER('categoria'), COMMA, COUNT, LPAREN, STAR, RPAREN, FROM, IDENTIFIER('ventas'), GROUP, BY, IDENTIFIER('categoria'), SEMICOLON, EOF]`
2. `Parser._parse_select()`
→ `_parse_select_list()`
→ `_parse_select_expr()` → `{"kind": "COLUMN", "name": "categoria"}`
→ `COMMA`
→ `_parse_select_expr()` → `{"kind": "COUNT_STAR"}`
→ `_match(GROUP)` → `_expect(BY)` → `_expect(IDENTIFIER)` → `"categoria"`
→ `AST: {"type": "SELECT", "table": "ventas", "columns": [...], "group_by": "categoria"}`
3. `Engine._select(node)`
→ `cond is None` → `DBManager.select_all("ventas")`
→ `rows = [{"id": 1, "categoria": "A", "monto": 100}, {"id": 2, "categoria": "B", "monto": 200}, {"id": 3, "categoria": "A", "monto": 150}, ...]`
→ `group_by = "categoria"`
→ `_apply_group_by(rows, [...], "categoria")`
→ `groups = {"A": 3, "B": 2, "C": 1}`
→ `resultado = [{"categoria": "A", "COUNT(*)": 3}, {"categoria": "B", "COUNT(*)": 2}, {"categoria": "C", "COUNT(*)": 1}]`

Cuadro 28: Complejidad de Group By con External Hashing

Operación	Complejidad promedio
Particionamiento	$O(n)$
Conteo por grupo	$O(n)$
Espacio temporal	$O(n)$

8.4. Replacement Selection

Replacement Selection es una optimización del ordenamiento externo que permite generar runs más largos durante la primera fase del External Sort. Su objetivo es reducir la cantidad de runs iniciales y, por lo tanto, disminuir el número de pasadas de merge requeridas.

A diferencia del External Merge Sort estándar, que genera runs del tamaño aproximado del buffer disponible, Replacement Selection utiliza un min-heap con particionamiento activo y frío en memoria principal. Gracias a ello, los runs generados pueden ser, en promedio, aproximadamente dos veces más grandes que la capacidad del buffer.

Idea principal

El algoritmo mantiene un heap con registros activos. En cada paso se extrae el menor registro y se escribe en el run actual. Luego, se lee un nuevo registro desde el archivo de entrada. Si este nuevo registro mantiene el orden del run actual, se inserta nuevamente como activo. Si rompe el orden, se marca como frío y se reserva para el siguiente run.

Cuando ya no quedan registros activos, el run actual termina y los registros fríos se reactivan para iniciar un nuevo run.

Procedimiento

1. Cargar registros iniciales en un min-heap.
2. Extraer el menor registro y escribirlo en el run actual.
3. Leer el siguiente registro del archivo de entrada.
4. Si el nuevo registro es mayor o igual al último escrito, se mantiene en el heap activo.
5. Si el nuevo registro es menor, se marca como frío para el siguiente run.
6. Cuando no quedan registros activos, se cierra el run actual.
7. Los registros fríos se reactivan para construir el siguiente run.

Integración con SQL

Se integró al motor SQL mediante un trigger por comentario:

—replacement

Esto permite comparar el rendimiento entre Merge Sort estándar y Replacement Selection sin modificar la gramática principal del parser.

Ventajas

- Genera runs más largos que External Sort estándar.
- Reduce la cantidad de archivos temporales.
- Disminuye las lecturas y escrituras durante la fase de merge.
- Permite procesar datasets más grandes que la memoria RAM disponible.
- Se integra con el frontend mediante badges dinámicos y estadísticas de I/O.

También se implementó una gestión explícita de file handles para garantizar compatibilidad multiplataforma entre Windows y Linux, y evitar fugas de memoria en el servidor.

Cuadro 29: Comparación entre External Sort estándar y Replacement Selection

Criterio	External Sort	Replacement Selection
Tamaño promedio de run	Aproximadamente tamaño del buffer	Aproximadamente 2 veces el tamaño del buffer
Cantidad de runs iniciales	Mayor	Menor
Costo de merge	Mayor	Menor
Uso de heap	No necesariamente	Sí, min-heap
Eficiencia en disco	Menor	Mayor

9. Conclusiones

El proyecto permitió implementar un motor de base de datos funcional basado completamente en estructuras de memoria secundaria, integrando componentes fundamentales de un DBMS moderno como parser SQL, administración de páginas, índices y algoritmos externos.

Se logró comprobar experimentalmente cómo distintas estructuras de indexación presentan ventajas específicas dependiendo del tipo de operación realizada. Mientras algunas estructuras optimizan búsquedas exactas, otras presentan mejor comportamiento para consultas espaciales o búsquedas por rango.

Asimismo, el desarrollo permitió comprender la importancia del manejo eficiente de disco, del buffer pool y de la organización física de datos para reducir costos de I/O y mejorar el rendimiento general del sistema.

Finalmente, el proyecto replica conceptos utilizados en sistemas reales de bases de datos, integrando almacenamiento físico, parsing, concurrencia y estructuras avanzadas de indexación dentro de una misma arquitectura.

10. Referencias

Metaweia, A. (2026). *Airbnb Open Data - New York City Listings*. Kaggle.

<https://www.kaggle.com/datasets/ahmed1metawea/airbnb-open-data-new-york-city-listings-20192022>

Readysset. (s. f.). *Readysset — Drop-in SQL Caching for PostgreSQL and MySQL*.
<https://readysset.io/blog/introducing-hash-join-algorithm>

Winand, M. (2026). *Partial Objects in ORMs: loading a subset of properties*. Use The Index, Luke.
<https://use-the-index-luke.com/sql/join/hash-join-partial-objects>